

Missouri University of Science and Technology

Intro to Data Science
CS 3402 (Spring 2026)

Lecture 14 DNNs & Transformer

Xiaowei Yu, Ph.D.

Department of Computer Science

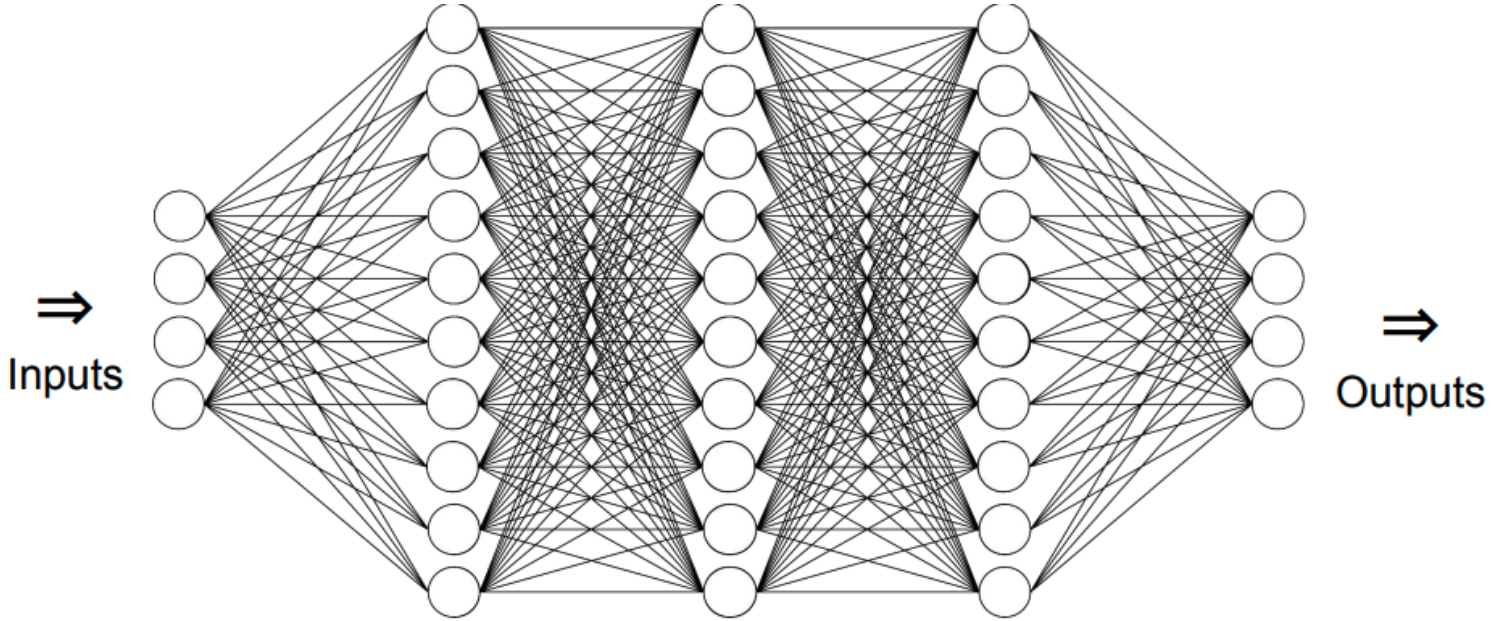
A Titular History of DNNs

- Why you should consider Neural Networks. 2022
- An Introduction to Neural Networks. 2023
- I don't need to convince you to use Neural Networks. 2024
- You Already use Neural Networks. 2025

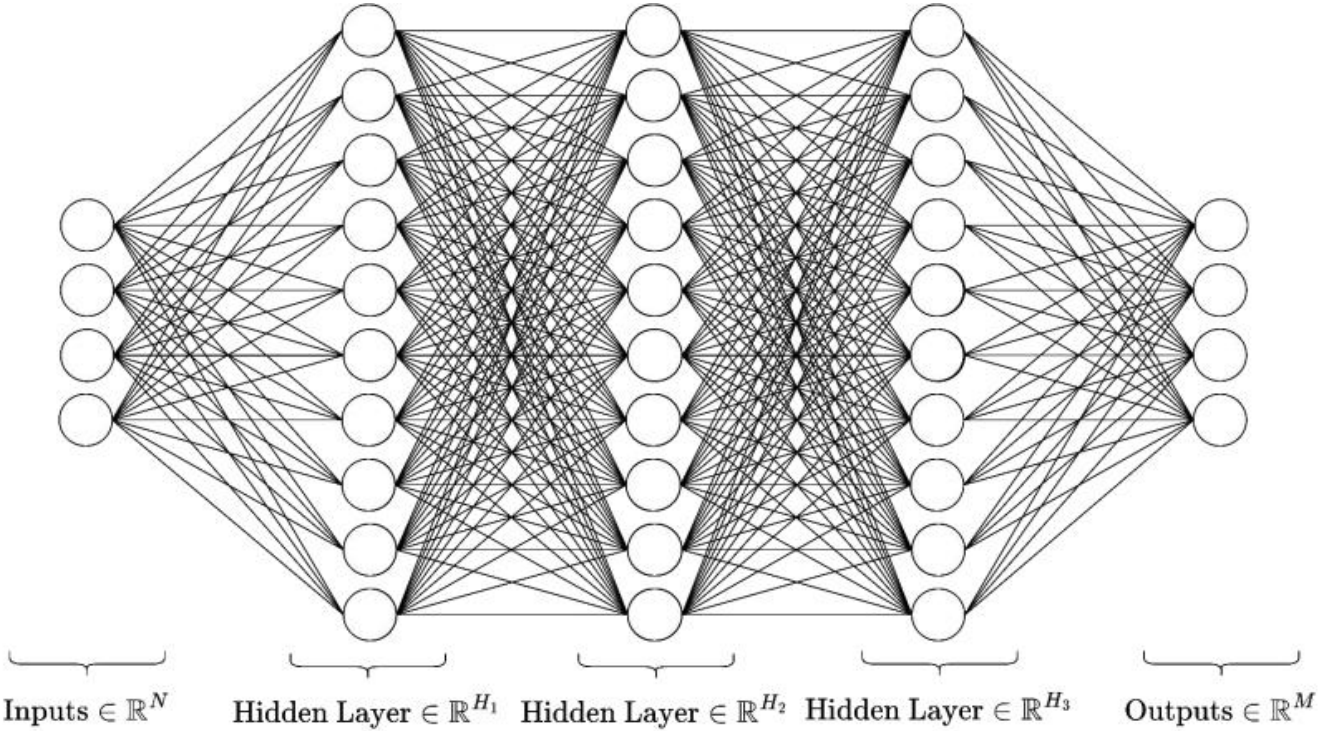
Common DNNs

- Multi-layer Perceptron (MLP)
- Convolutional Neural Networks (CNNs)
- Graph Neural Networks (GNN)
- Transformers
- …

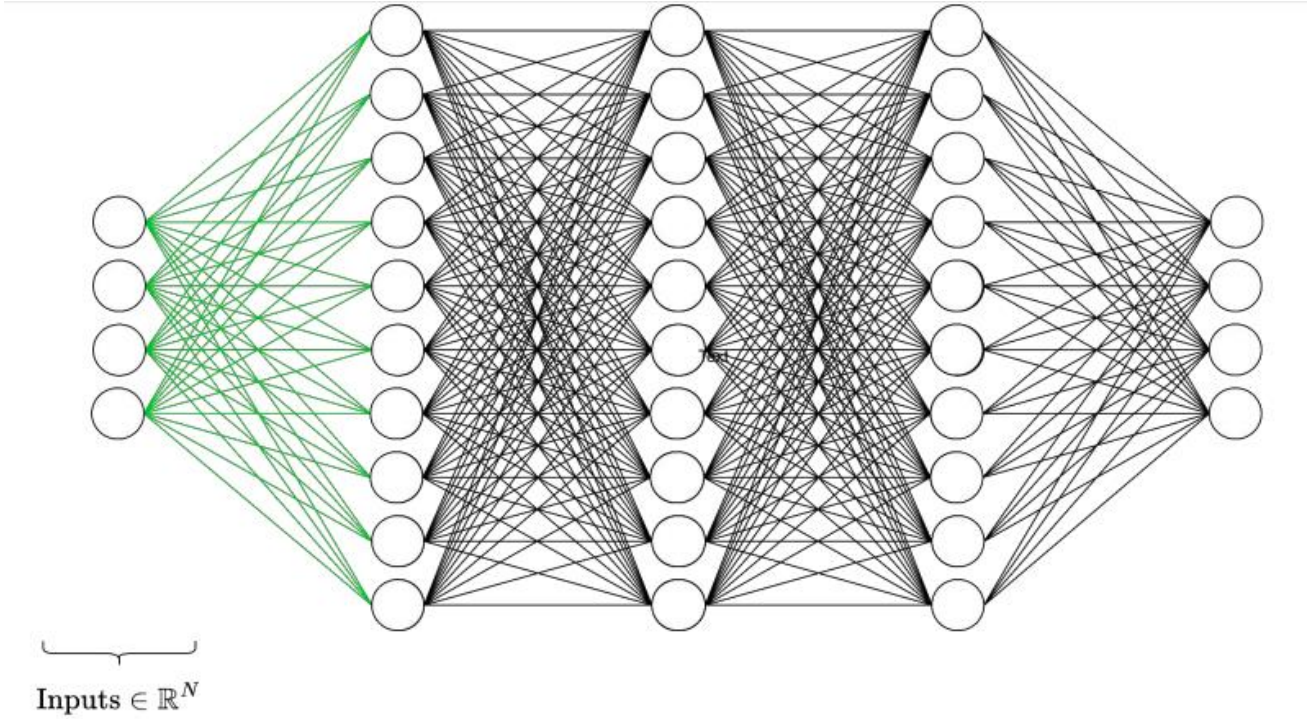
Multi-layer Perceptron (MLP)



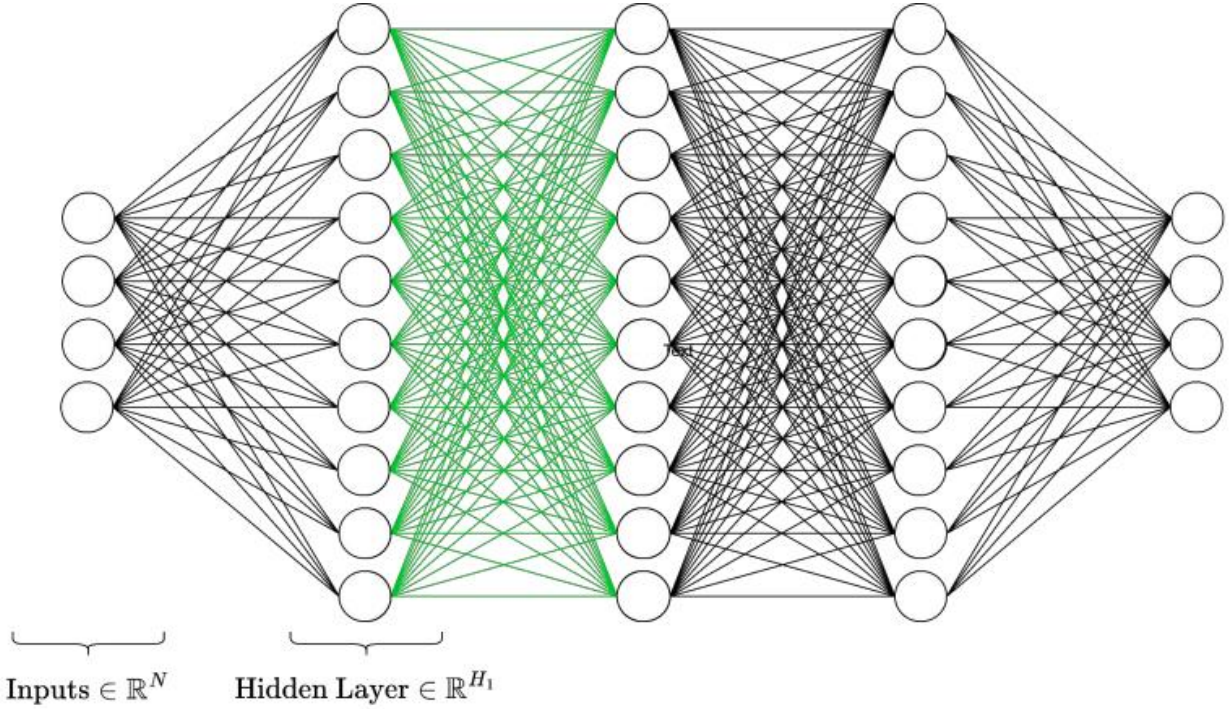
Multi-layer Perceptron (MLP)



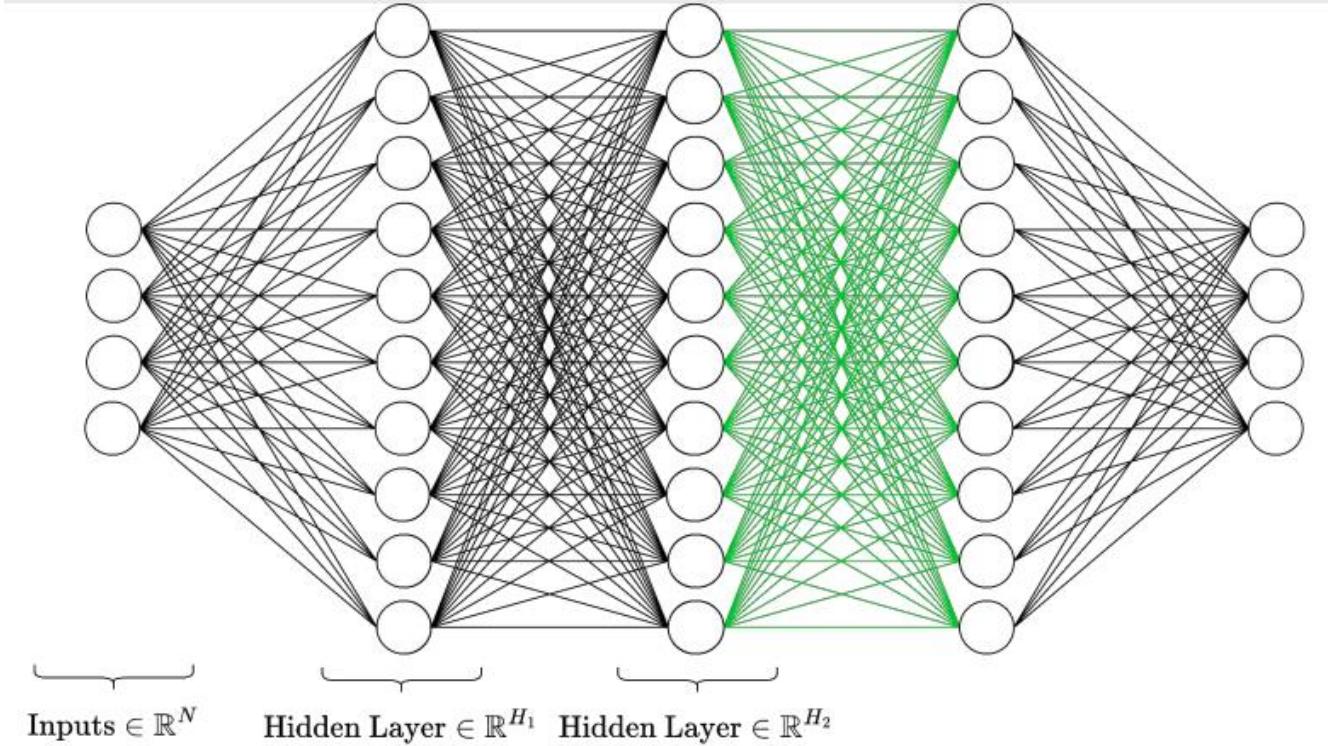
Multi-layer Perceptron (MLP)



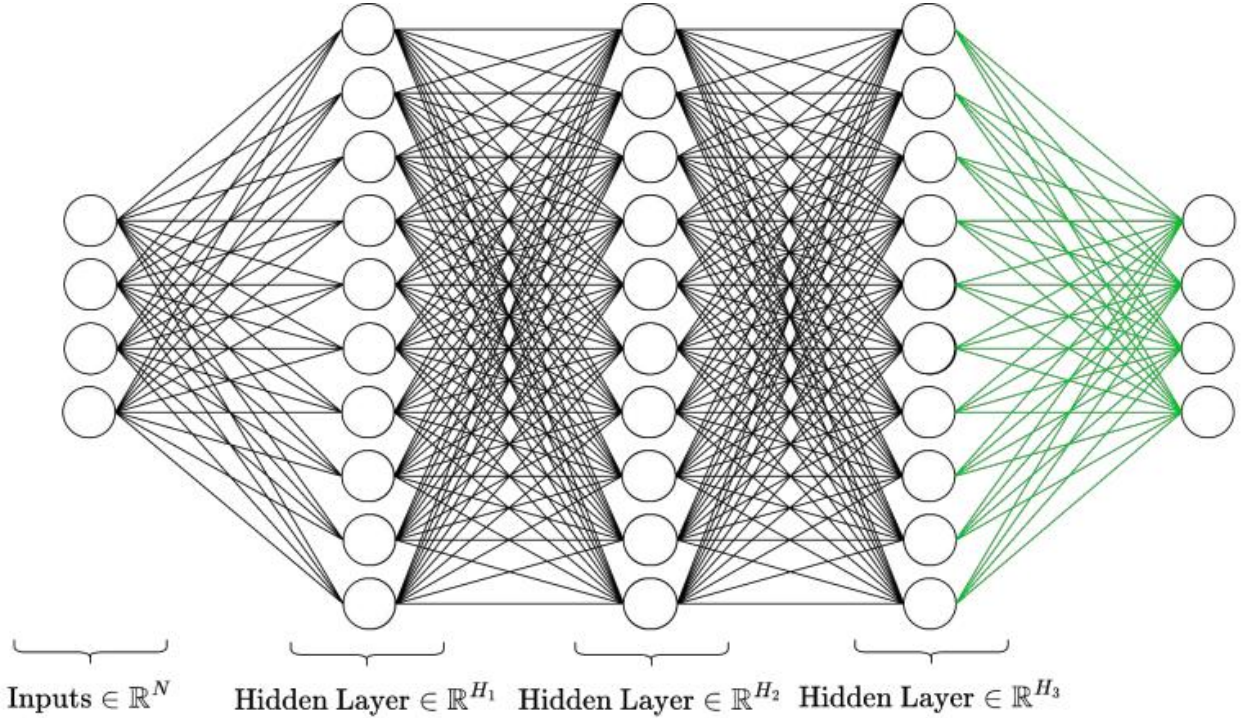
Multi-layer Perceptron (MLP)



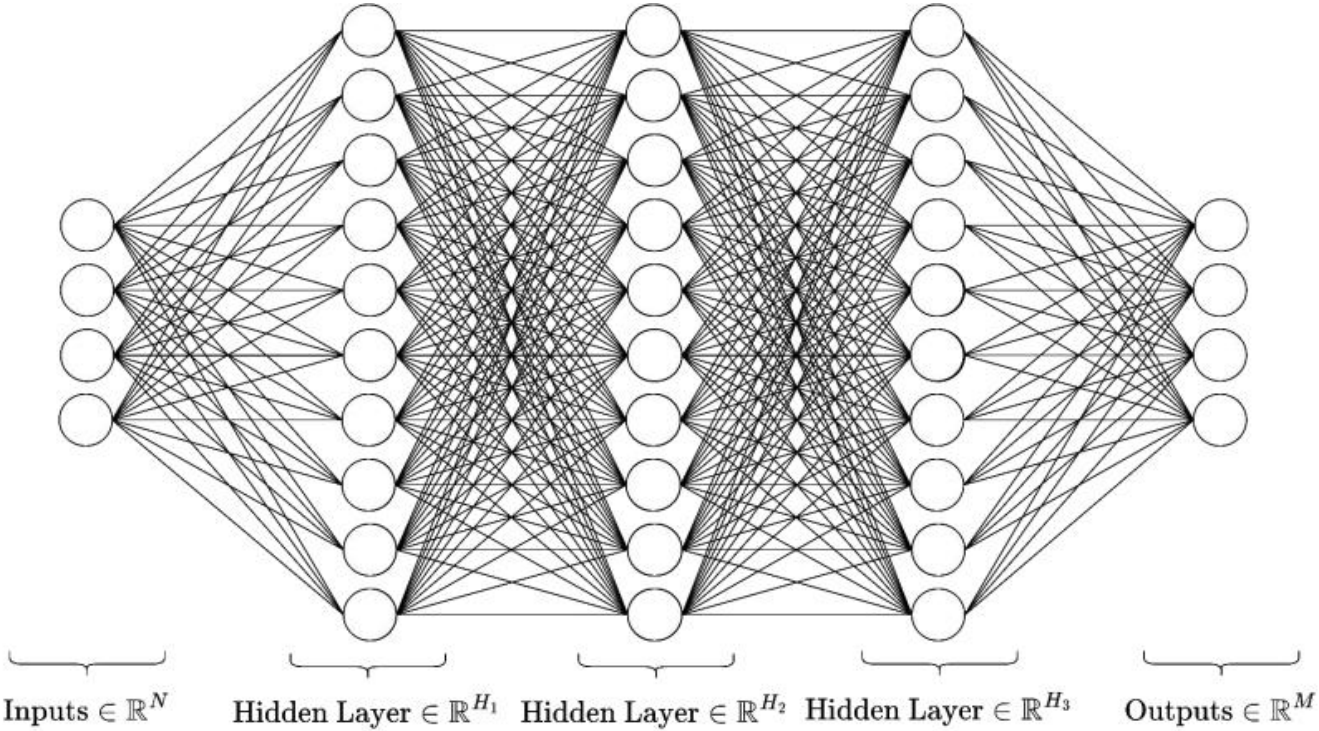
Multi-layer Perceptron (MLP)



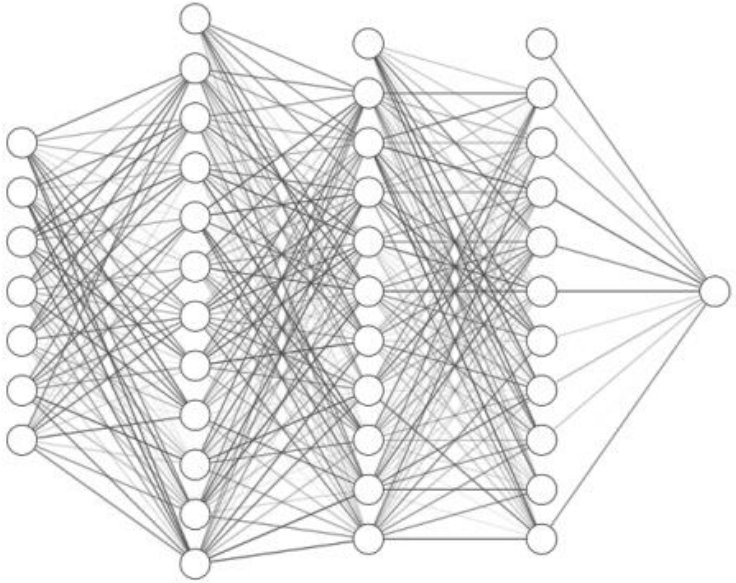
Multi-layer Perceptron (MLP)



Multi-layer Perceptron (MLP)



Multi-layer Perceptron (MLP)



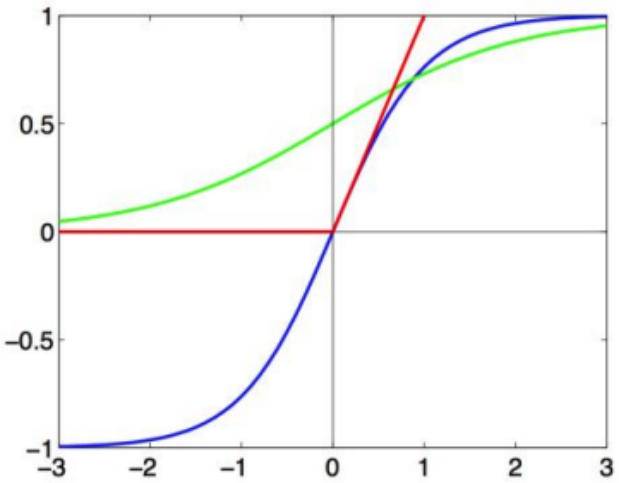
A multi-layer perceptron is a series of affine transformations of an input vector, each of which is wrapped in a non-linear activation function.

$$\mathcal{N} : \mathbb{R}^N \rightarrow \mathbb{R}^M$$
$$N, M \in \mathbb{N}$$

(*Translation:* an MLP is a fancy function)

Nonlinear Functions

Common nonlinear functions



$$\text{Sigmoid: } \sigma(x) = \frac{1}{1+e^{-x}}$$

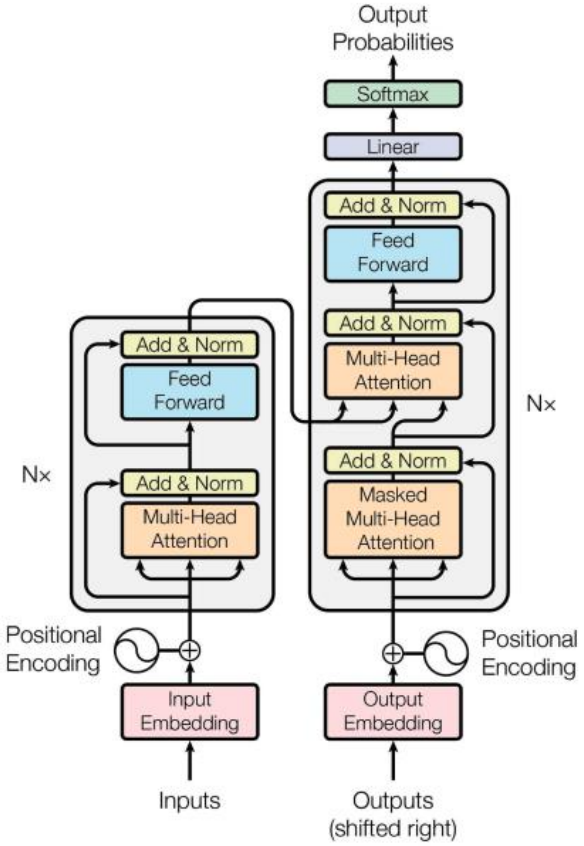
$$\text{Hyperbolic tangent: } \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$

Rectified Linear Unit:

$$\text{ReLU}(x) = max(0, x)$$

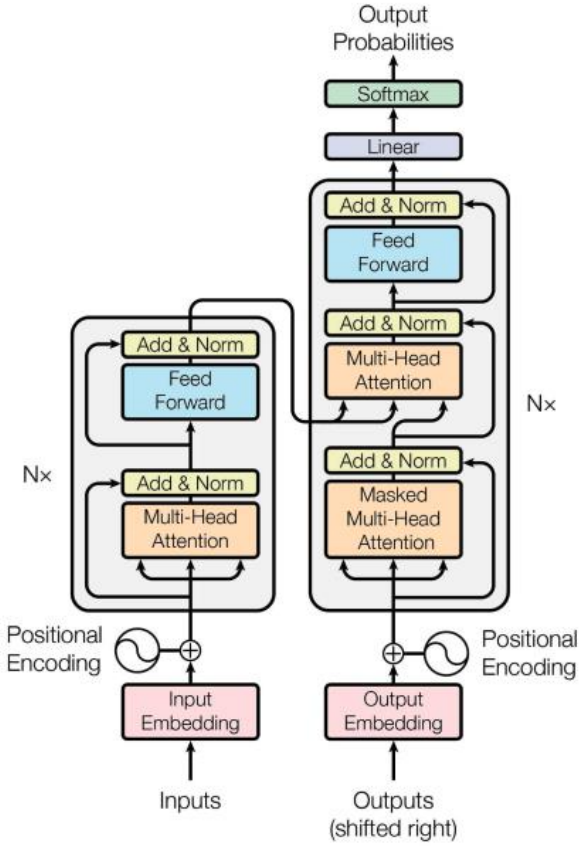
Transformers

Transformer Architecture



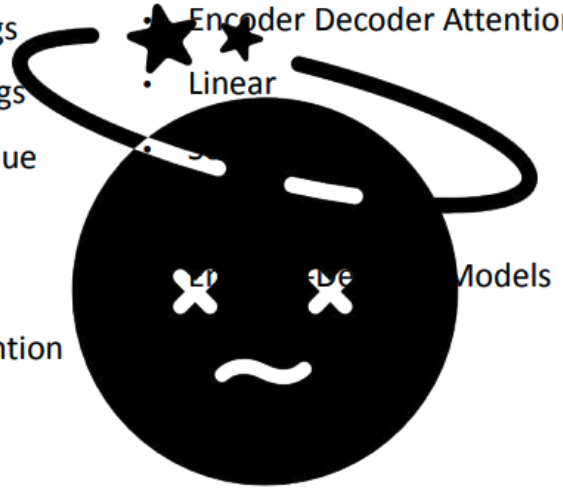
Transformers

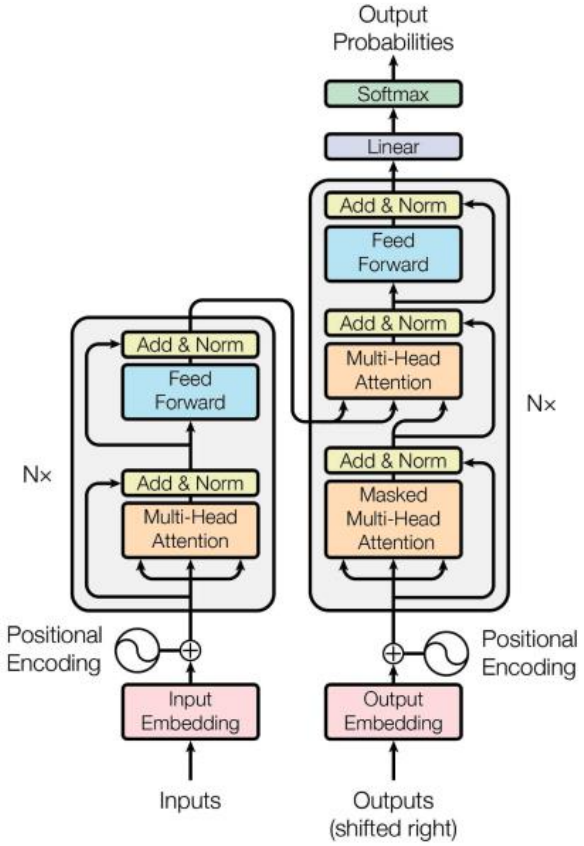
- Tokenization
- Input Embeddings
- Position Encodings
- Query, Key, & Value
- Attention
- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders
- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



Transformers

- Tokenization
- Input Embeddings
- Position Encodings
- Query, Key, & Value
- Attention
- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders
- Masked Attention
- Encoder Decoder Attention
- Linear
- [illegible] Models





Transformers

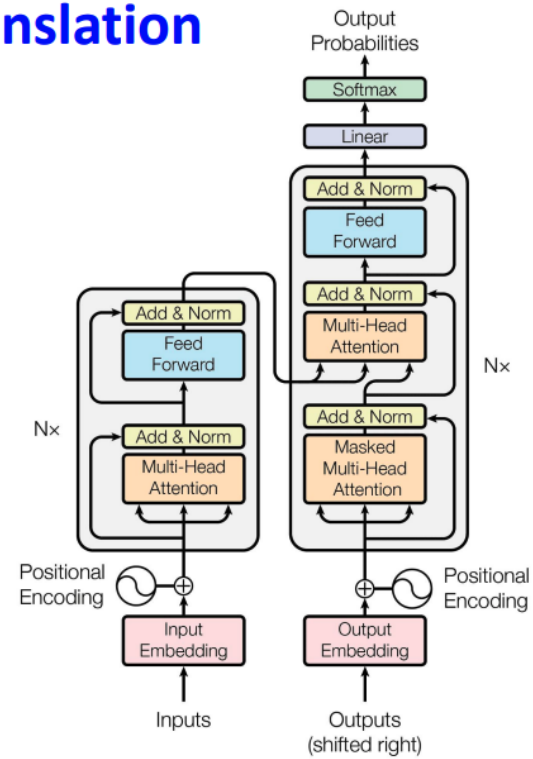
Machine Translation

Targets

Ich habe einen Apfel gegessen

Inputs

I ate an apple



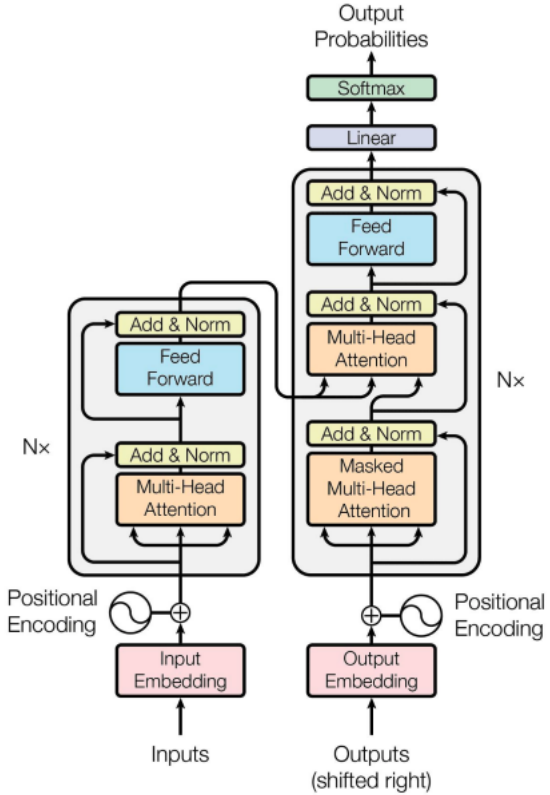
Transformers

Inputs

Processing Inputs

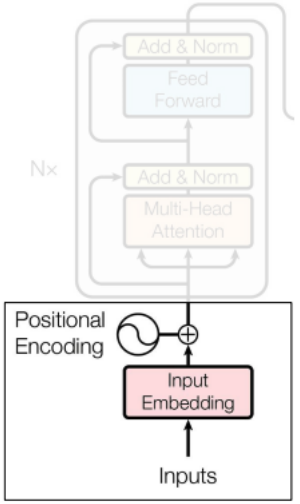
Inputs

I ate an apple



Transformers

Tokenization

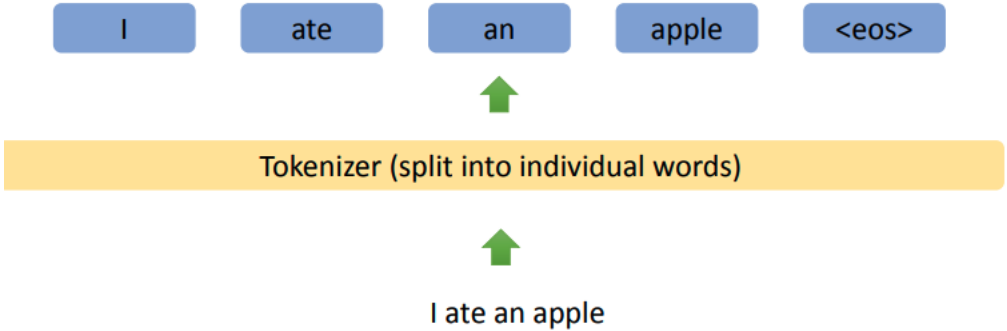


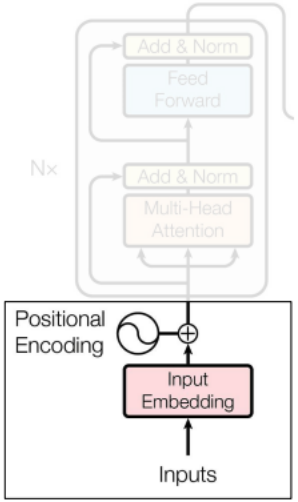
Tokenizer (split into individual words)

I ate an apple

Transformers

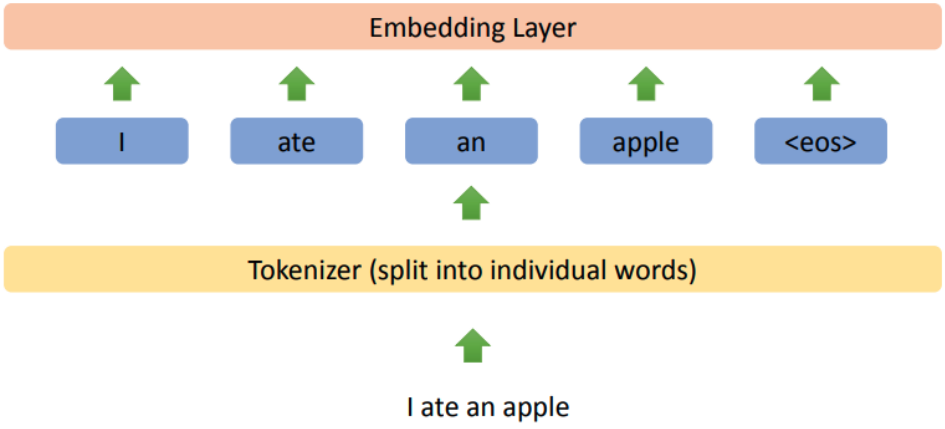
Tokenization

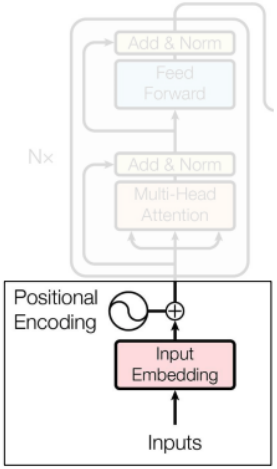




Transformers

Input Embeddings

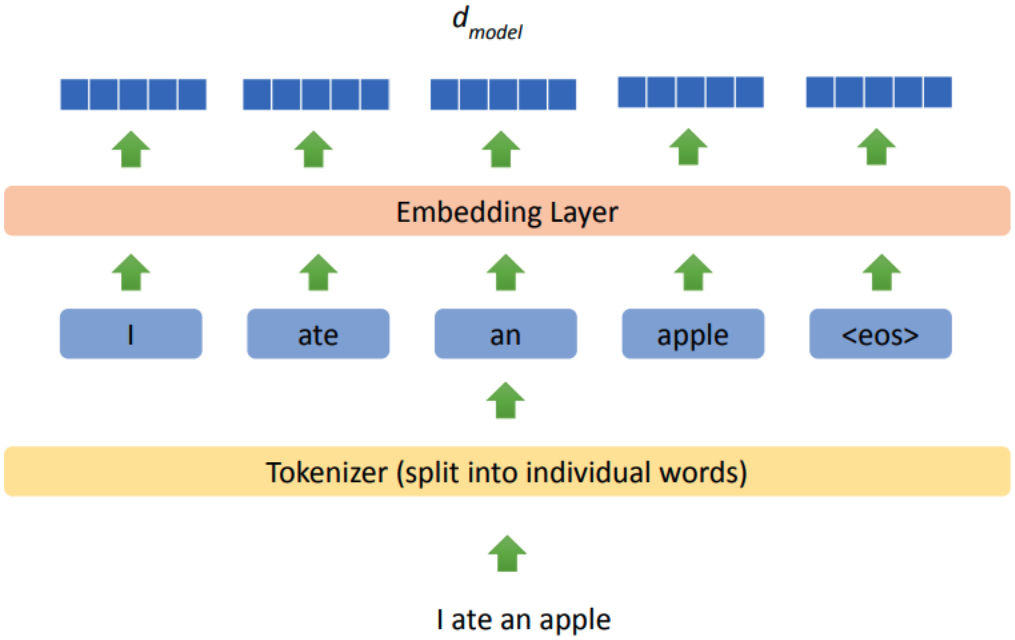


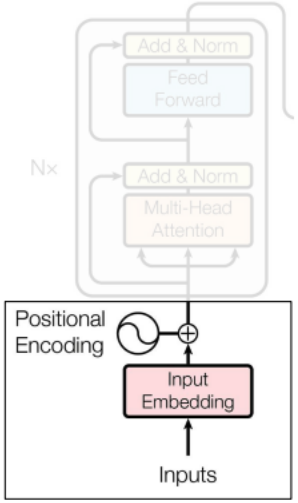


Generate Input Embeddings

Transformers

Input Embeddings



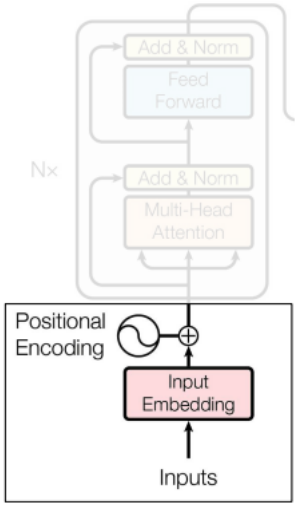


Generate Input Embeddings

Transformers

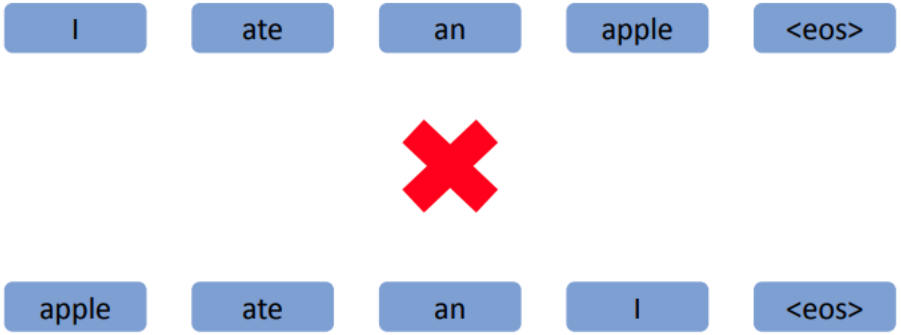
Position Encodings

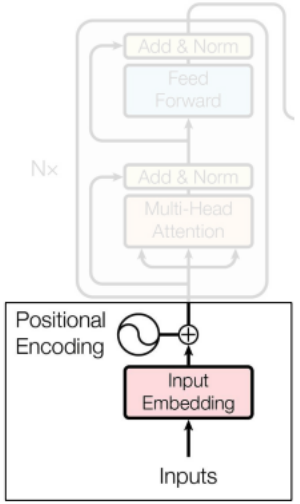
I | ate | an | apple | <eos>



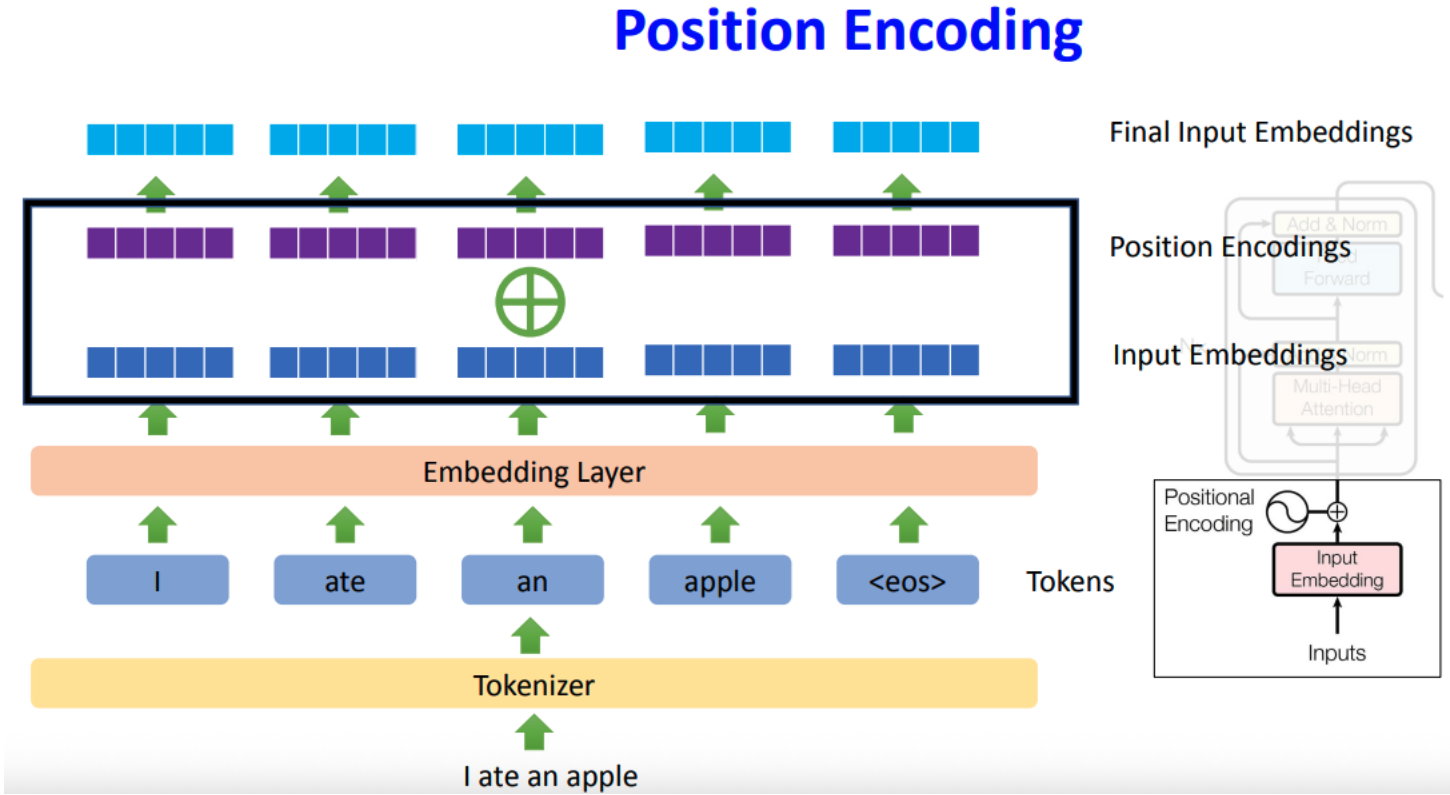
Transformers

Position Encodings





Transformers

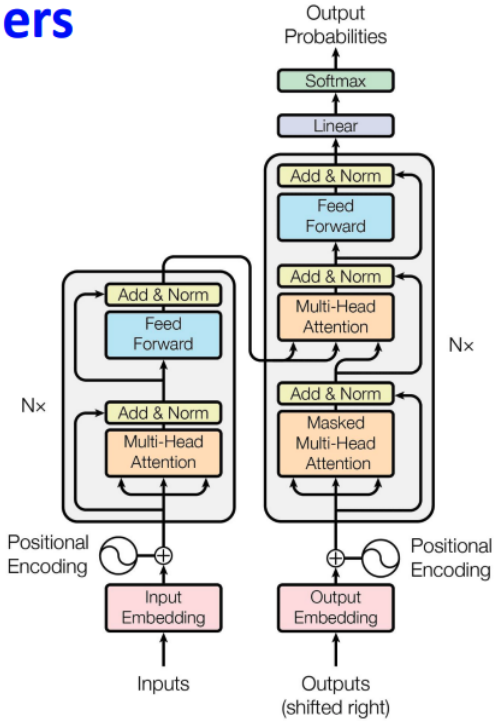


Transformers

Transformers

- ✔ Tokenization
- ✔ Input Embeddings
- ✔ Position Encodings
- Query, Key, & Value
- Attention
- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders

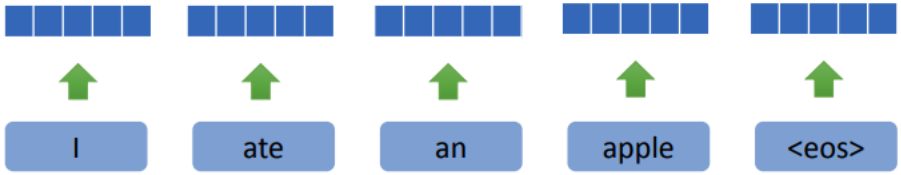
- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models

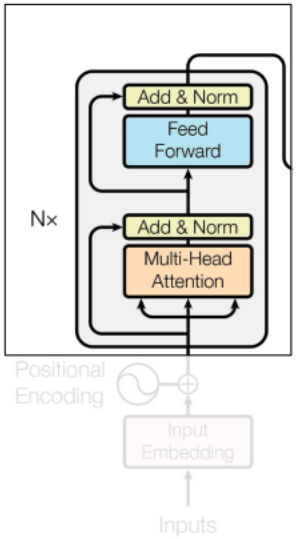


Transformers

Encoder

WHERE IS THE CONTEXT ?



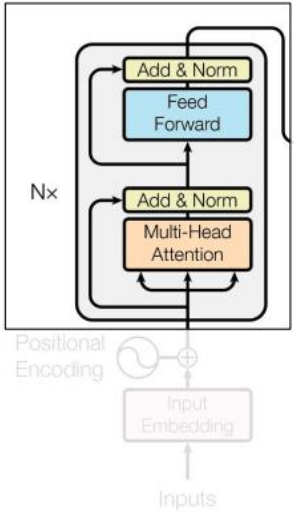


Transformers

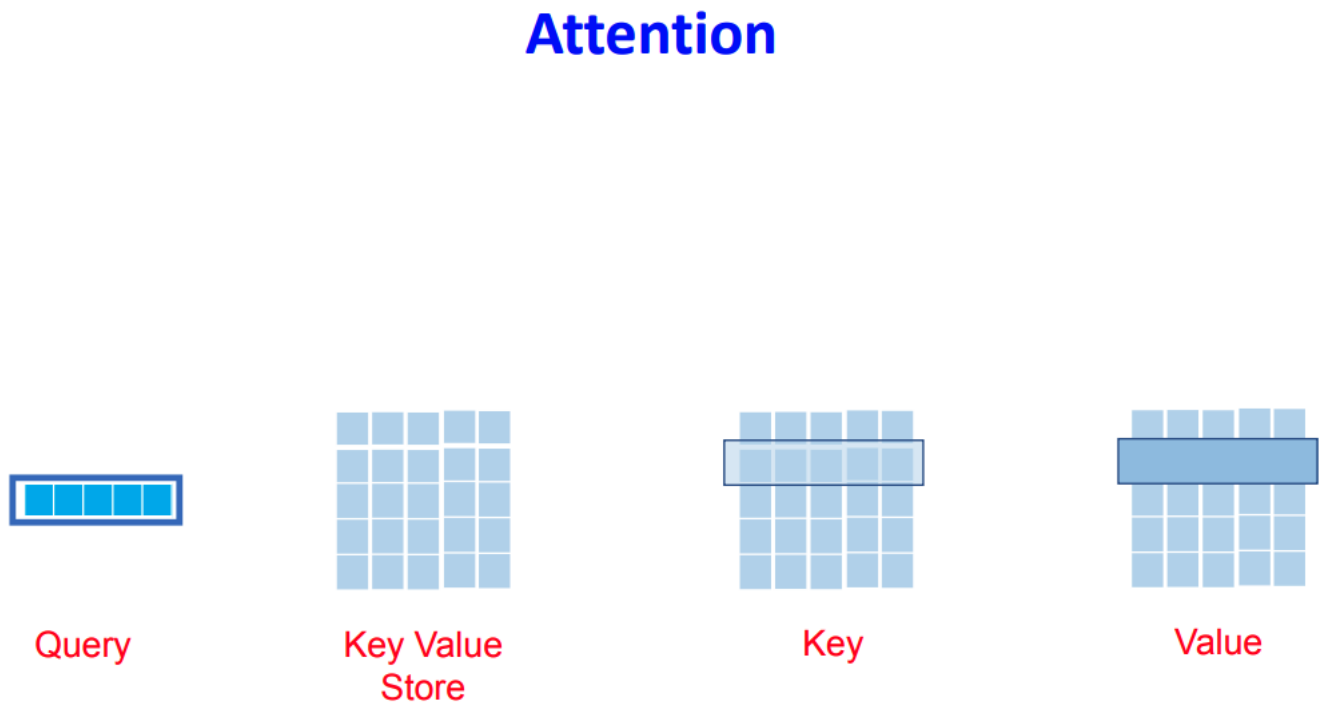
Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

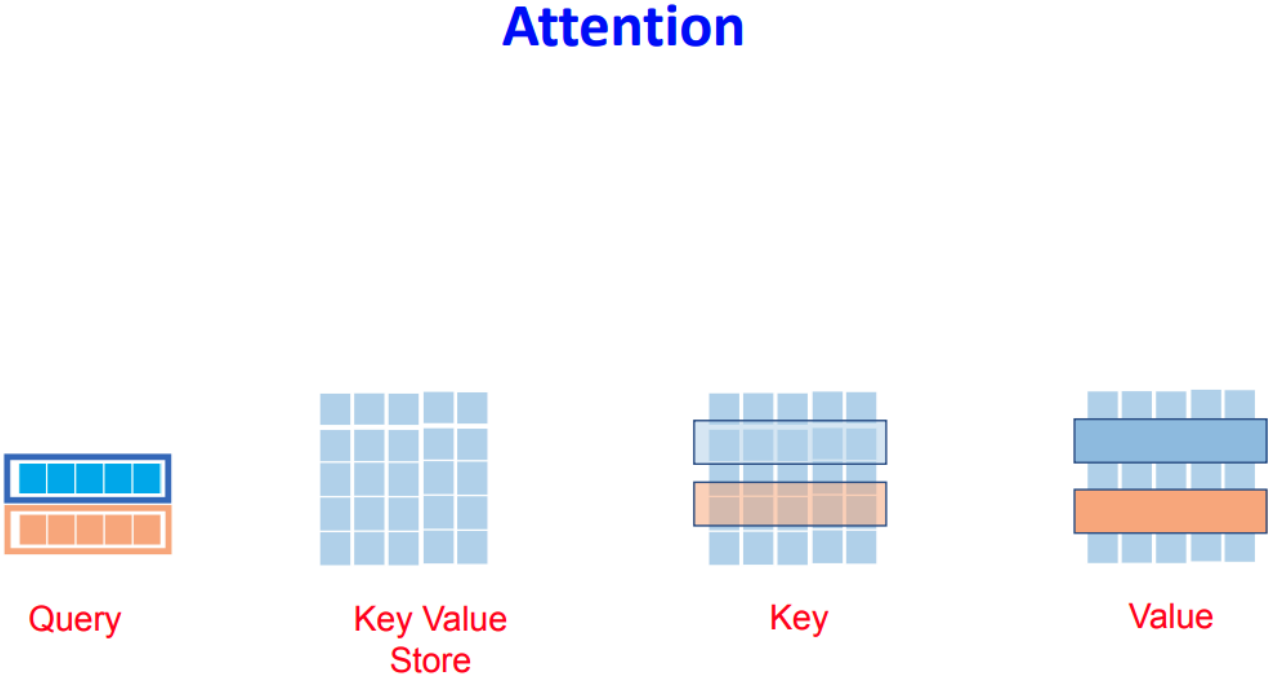
- Query
- Key
- Value



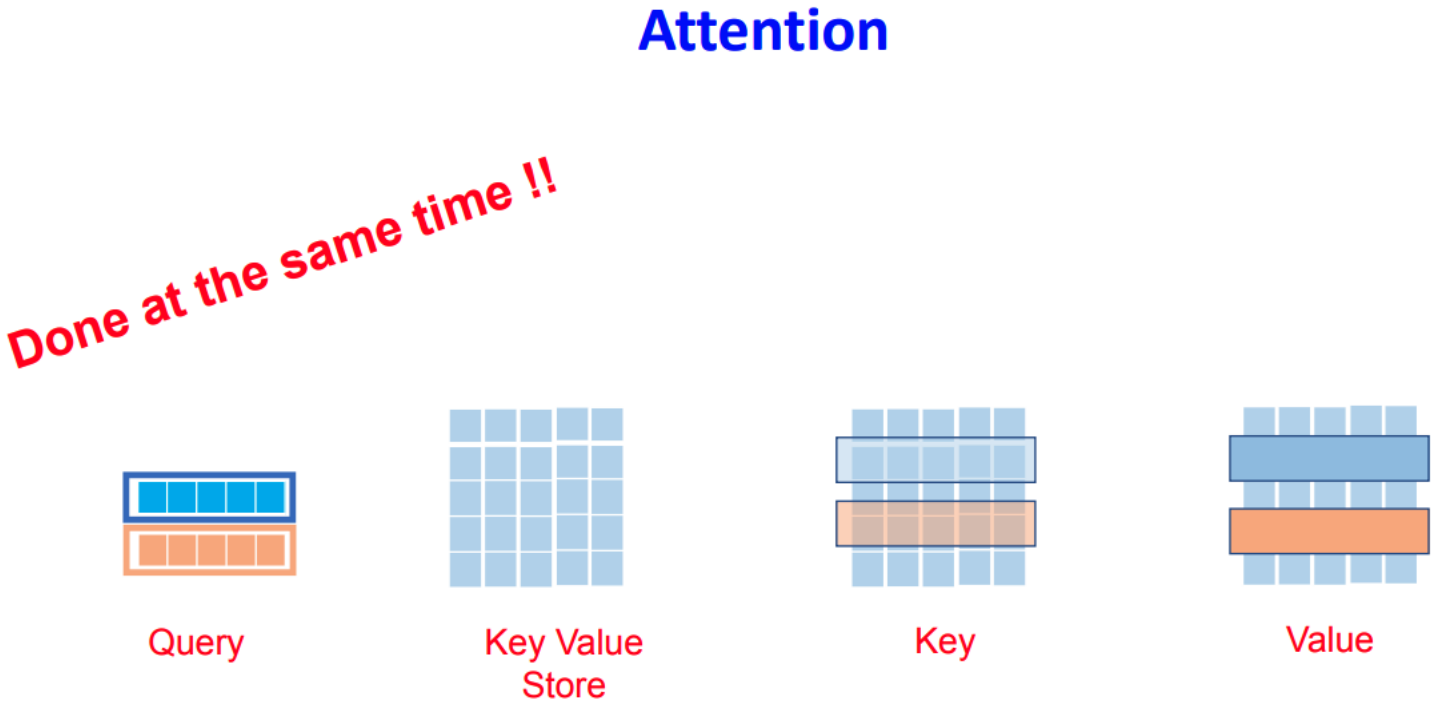
Attention



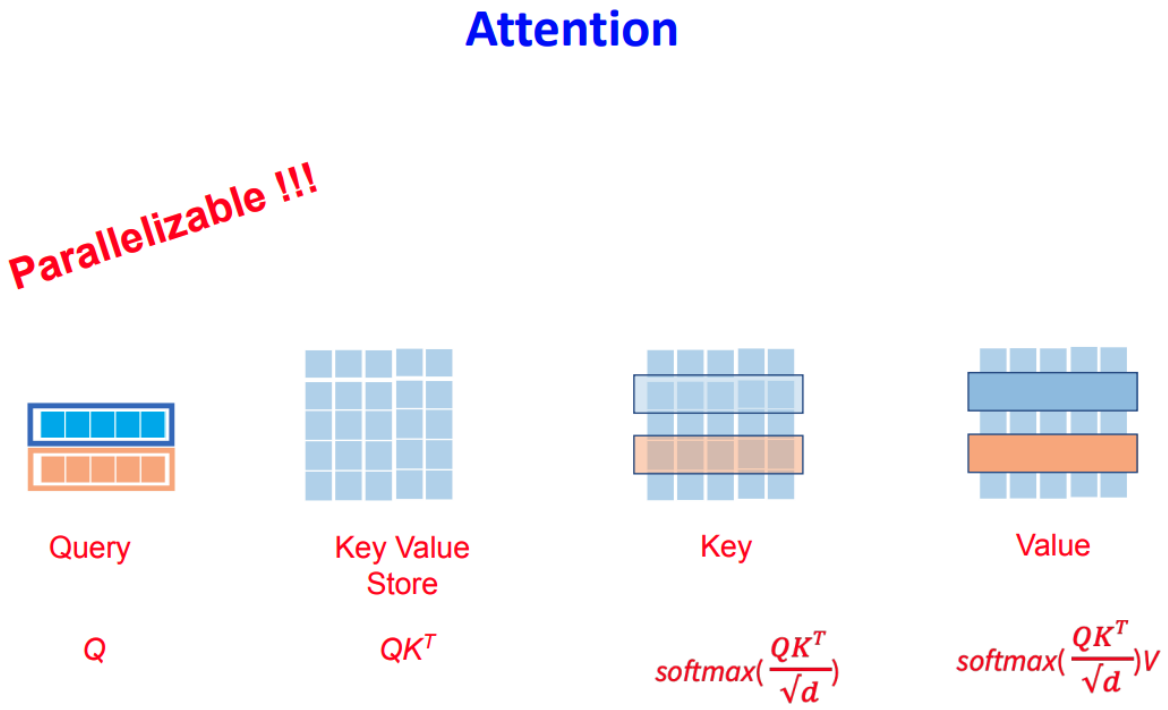
Attention



Attention

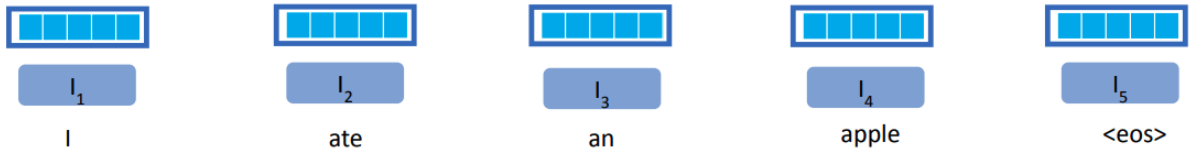

Attention
Done at the same time !!
Query
Key Value Store
Key
Value

Attention


Attention
Parallelizable !!!
Query
Key Value Store
Key
Value
Q
QK^T
$softmax(\frac{QK^T}{\sqrt{d}})$
$softmax(\frac{QK^T}{\sqrt{d}})V$

Attention

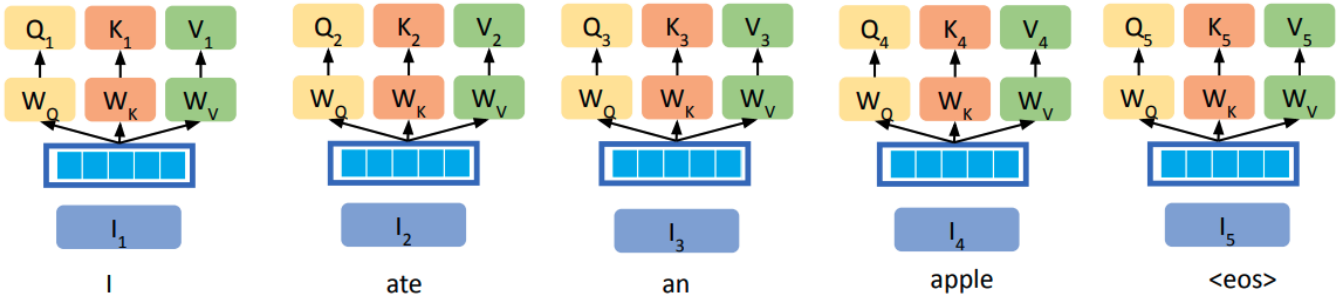
Attention



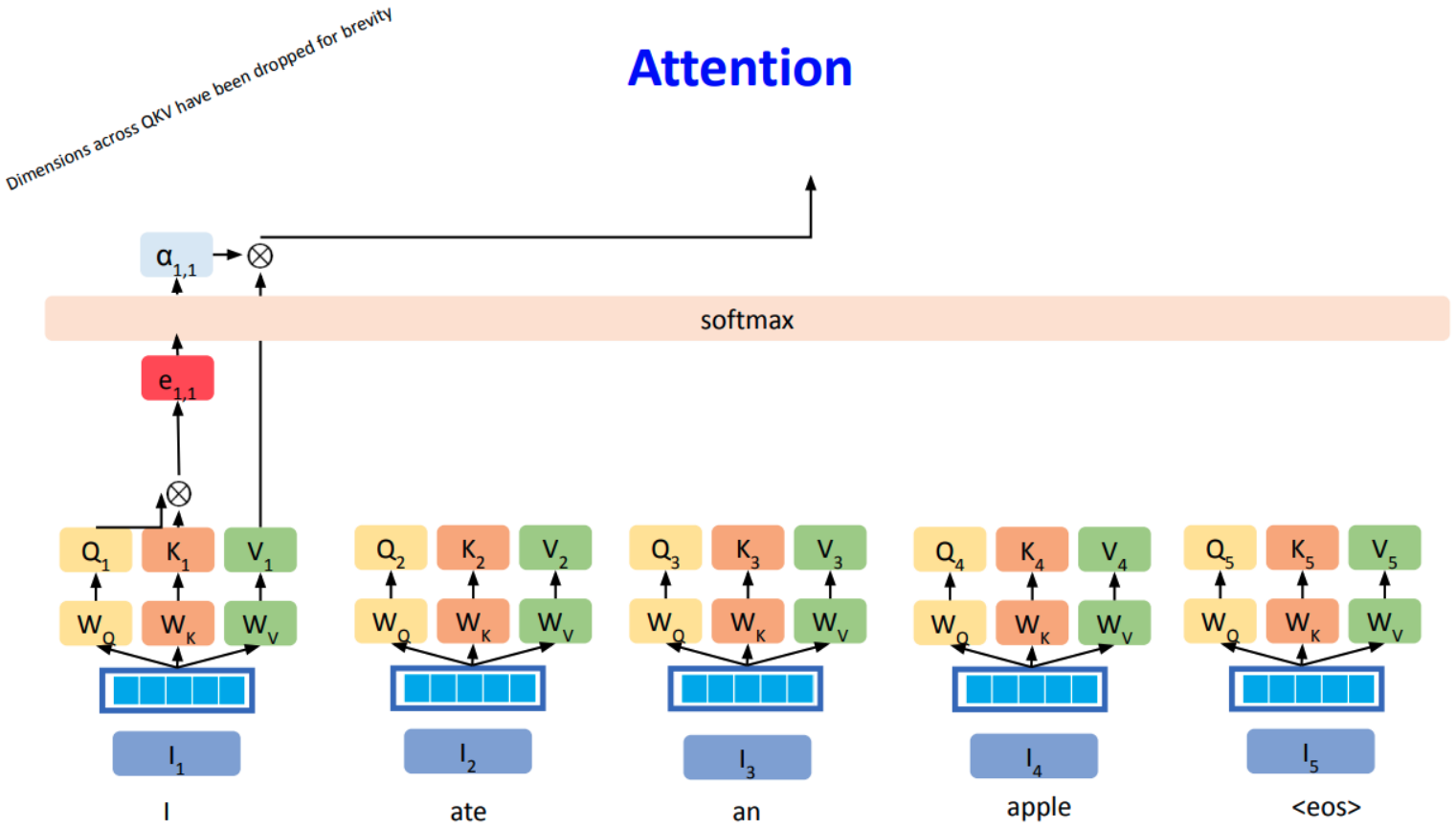
Attention

Dimensions across QKV have been dropped for brevity

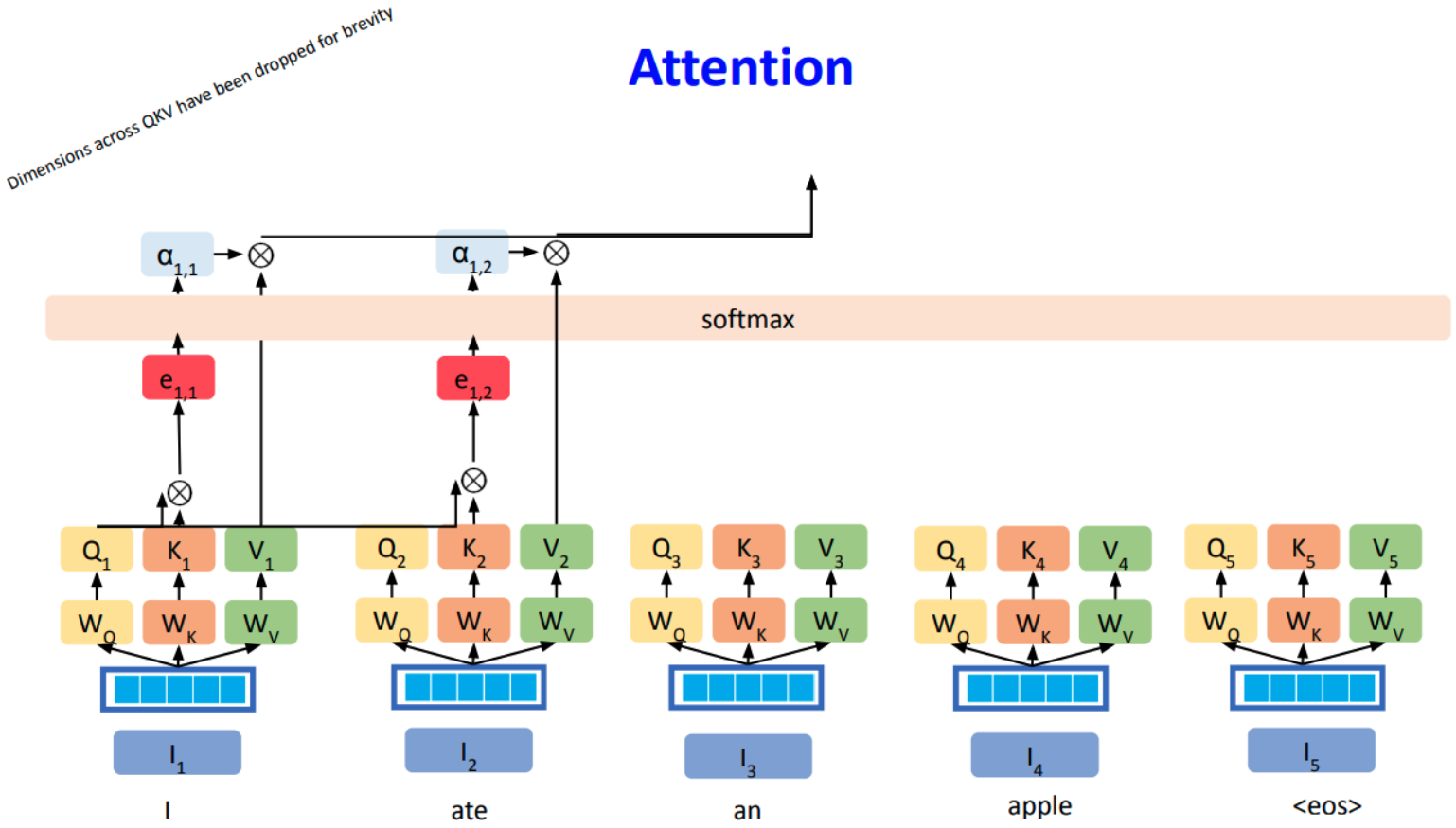
Attention



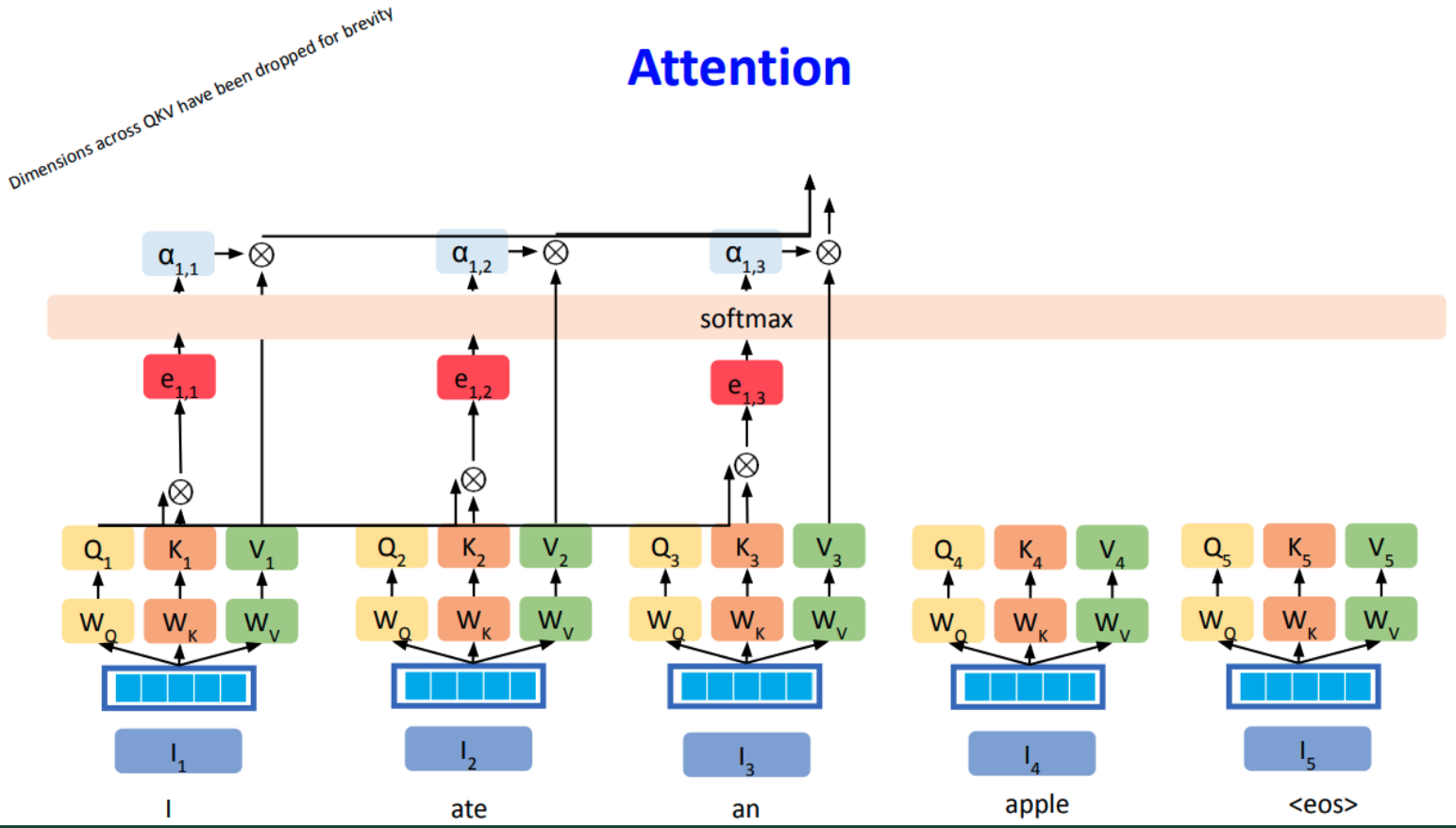
Attention


Attention
Dimensions across QKV have been dropped for brevity
α1,1
softmax
e1,1
Q1 K1 V1
W0 WK WV
I1
I
Q2 K2 V2
W0 WK WV
I2
ate
Q3 K3 V3
W0 WK WV
I3
an
Q4 K4 V4
W0 WK WV
I4
apple
Q5 K5 V5
W0 WK WV
I5
<eos>

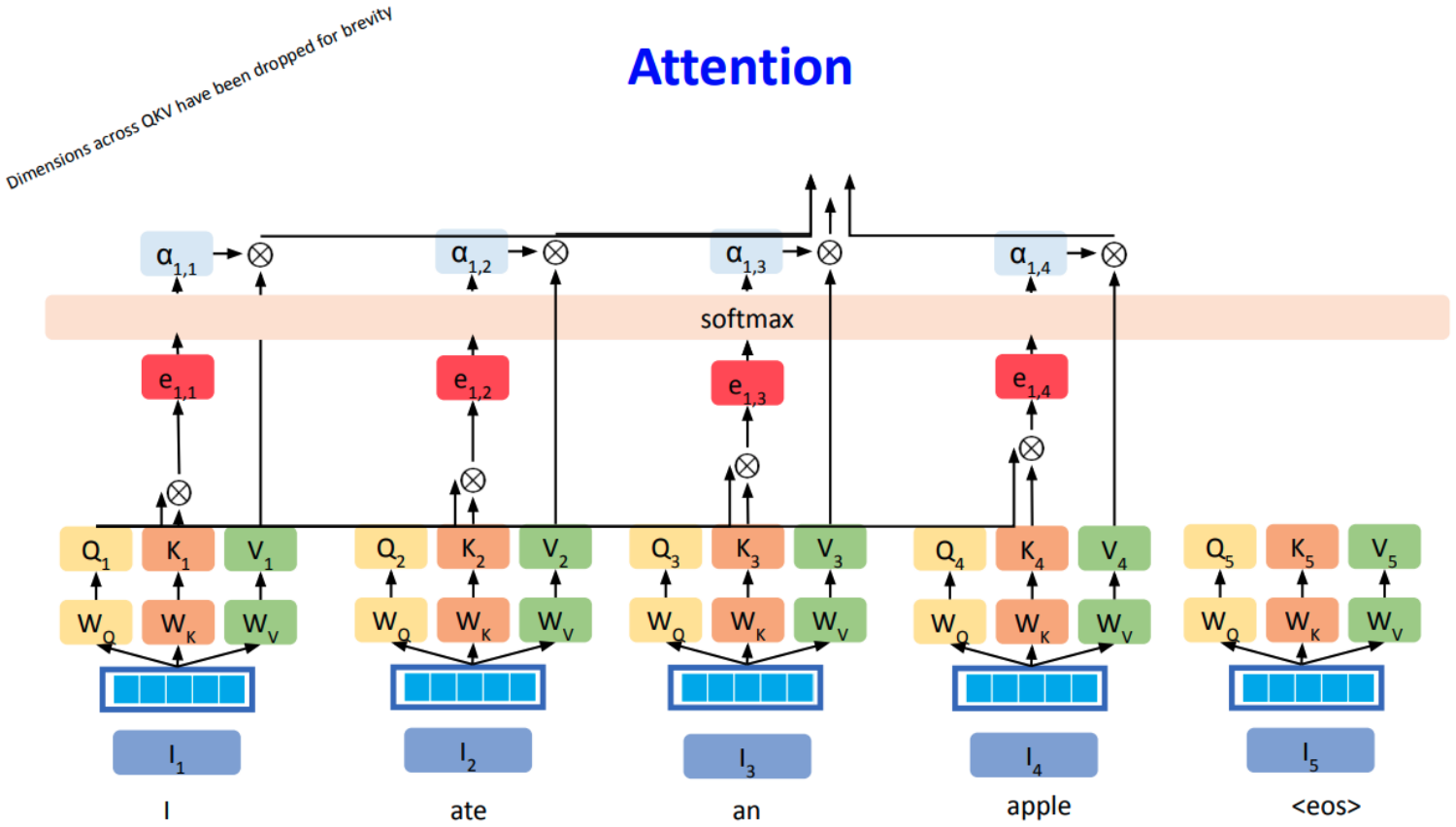
Attention


Attention
Dimensions across QKV have been dropped for brevity
softmax
I
ate
an
apple
<eos>

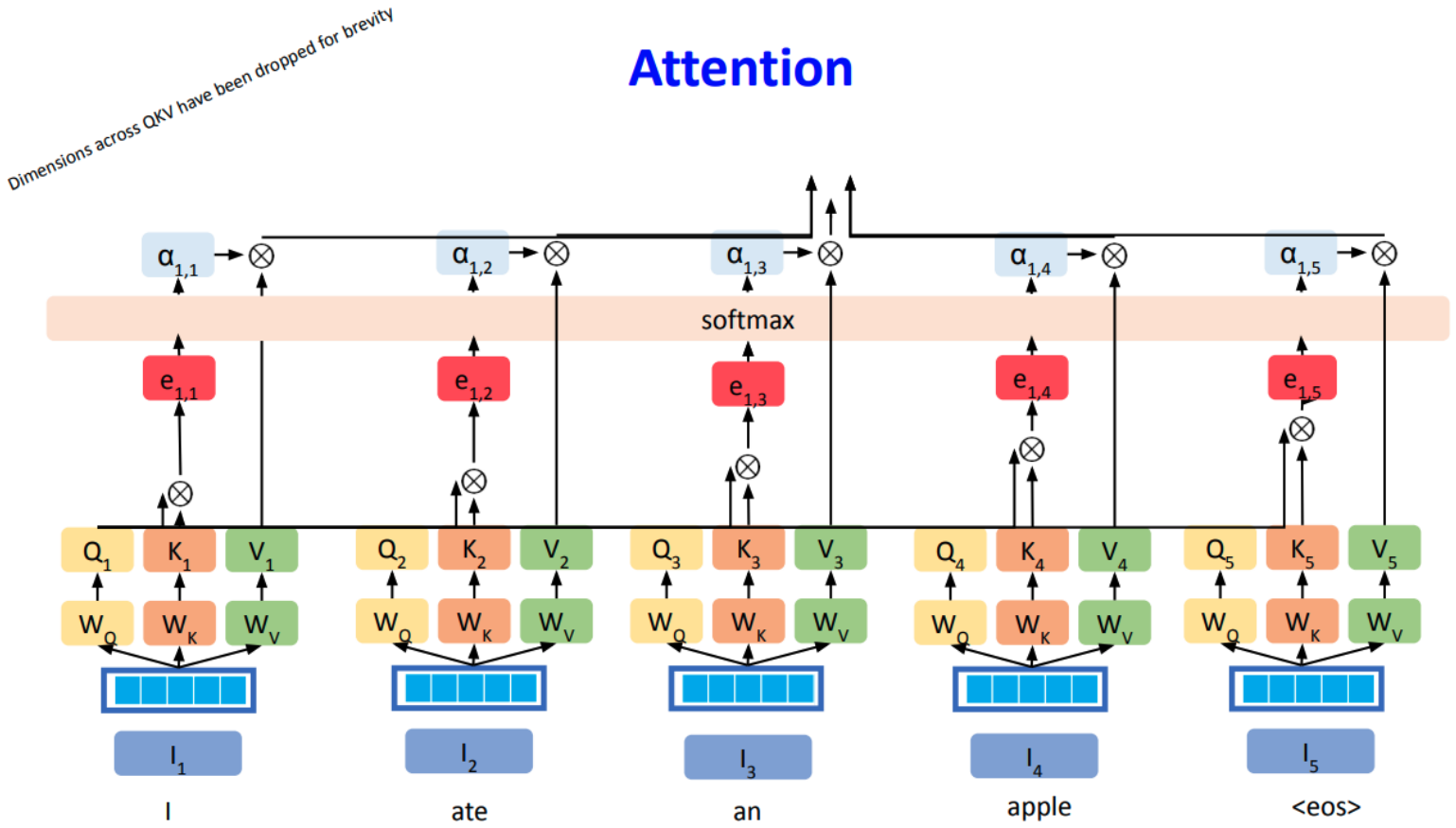
Attention


Attention
Dimensions across QKV have been dropped for brevity
softmax
I
ate
an
apple
<eos>

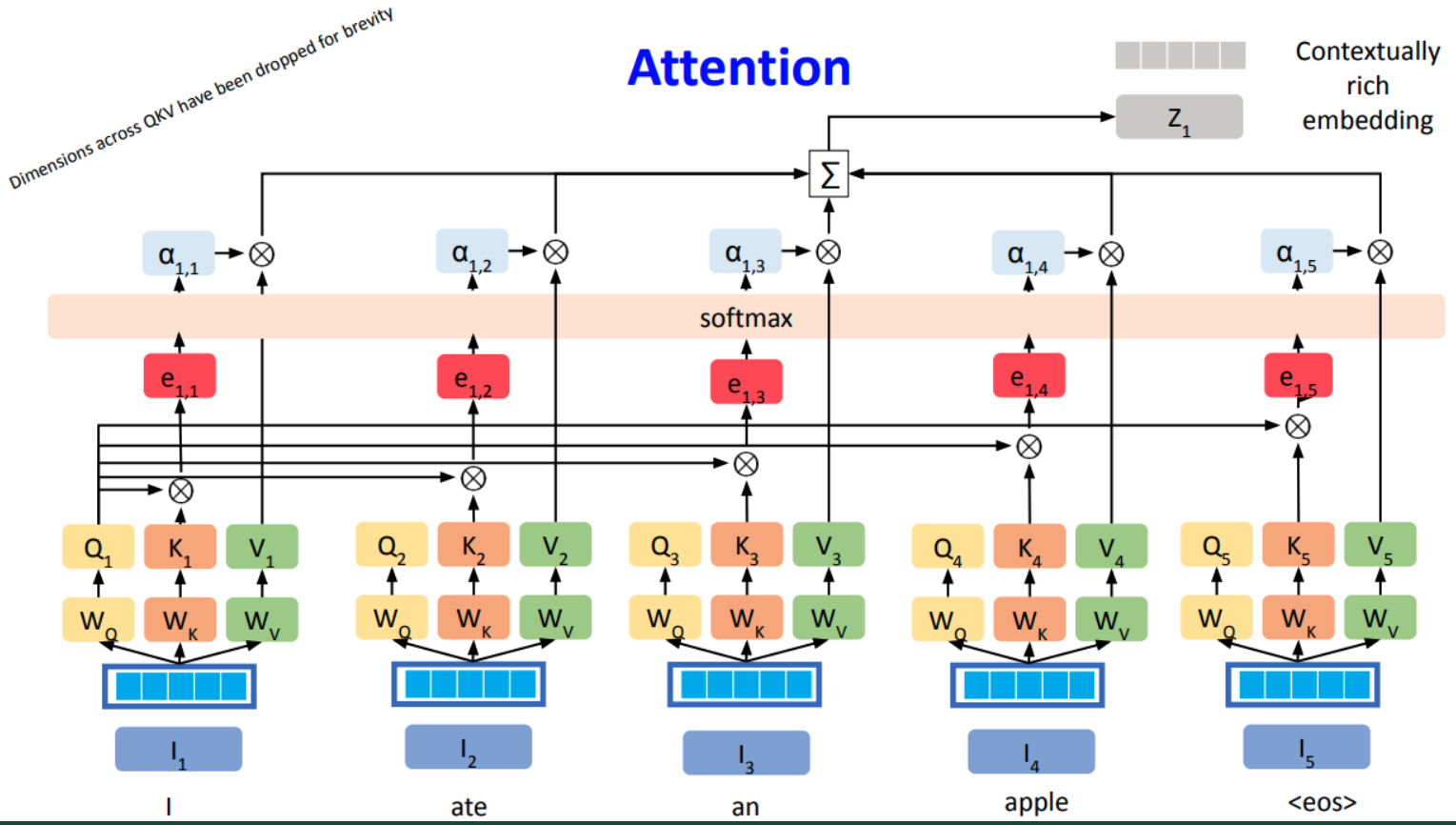
Attention


Attention
Dimensions across QKV have been dropped for brevity
softmax
I
ate
an
apple
<eos>

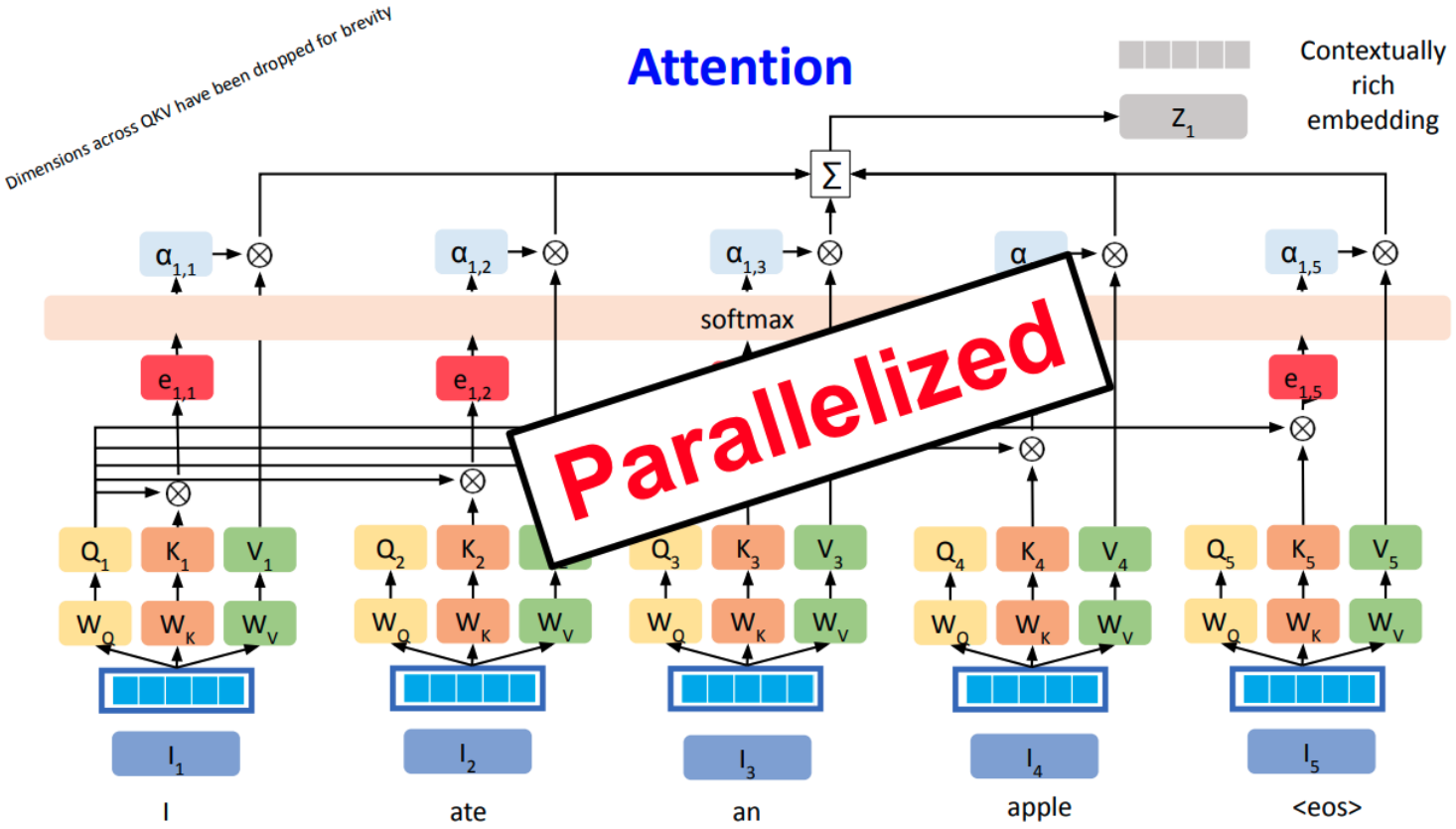
Attention


Attention
Dimensions across QKV have been dropped for brevity
softmax
I
ate
an
apple
<eos>

Attention


Attention
Dimensions across QKV have been dropped for brevity
Contextually rich embedding
Z_1
Σ
$\alpha_{1,1}$ $\alpha_{1,2}$ $\alpha_{1,3}$ $\alpha_{1,4}$ $\alpha_{1,5}$
softmax
$e_{1,1}$ $e_{1,2}$ $e_{1,3}$ $e_{1,4}$ $e_{1,5}$
Q_1 K_1 V_1
Q_2 K_2 V_2
Q_3 K_3 V_3
Q_4 K_4 V_4
Q_5 K_5 V_5
W_Q W_K W_V
I_1 I_2 I_3 I_4 I_5
I
ate
an
apple
<eos>

Attention

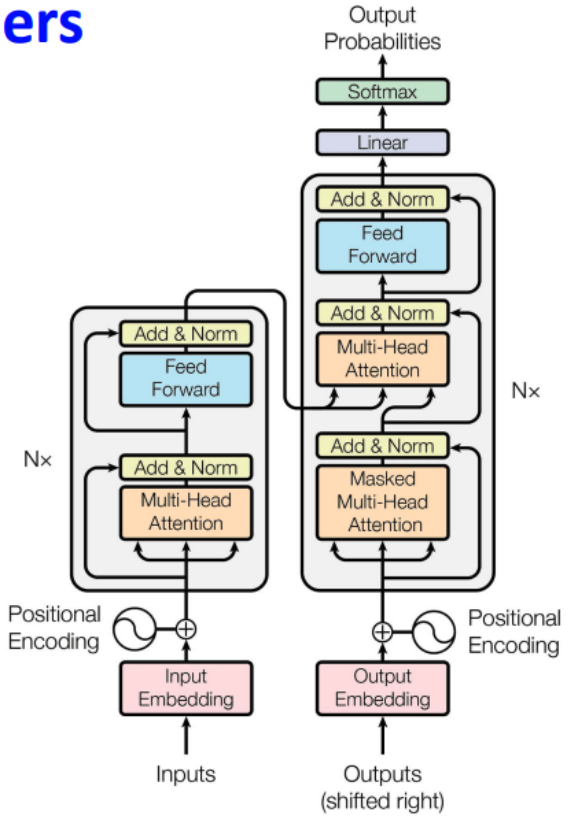

Dimensions across QKV have been dropped for brevity
Attention
Contextually rich embedding
Z_1
Σ
$\alpha_{1,1}$
$\alpha_{1,2}$
$\alpha_{1,3}$
α
$\alpha_{1,5}$
softmax
Parallelized
$e_{1,1}$
$e_{1,2}$
$e_{1,5}$
Q_1 K_1 V_1
Q_2 K_2
Q_3 K_3 V_3
Q_4 K_4 V_4
Q_5 K_5 V_5
W_Q W_K W_V
I_1
I_2
I_3
I_4
I_5
I
ate
an
apple
<eos>

Transformers

Transformers

- ✔ Tokenization
- ✔ Input Embeddings
- ✔ Position Encodings
- ✔ Query, Key, & Value
- ✔ Attention
  - Self Attention
  - Multi-Head Attention
  - Feed Forward
  - Add & Norm
  - Encoders

- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



Poll

Which of the following are true about attention?

a. To calculate attention weights for input I2 , you would use key k2 , and all queries

b. To calculate attention weights for input I2 , you would use query q2 , and all keys

c. We scale the QKT product to bring attention weights in the range of [0,1]

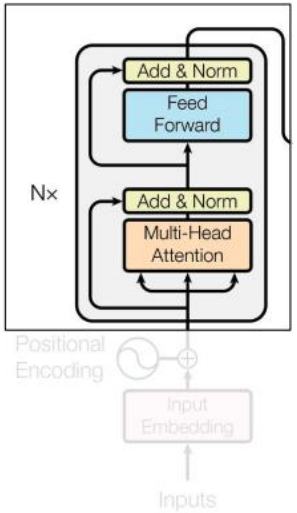
d. We scale the QKT product to allow for numerical stability

Self-Attention

Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

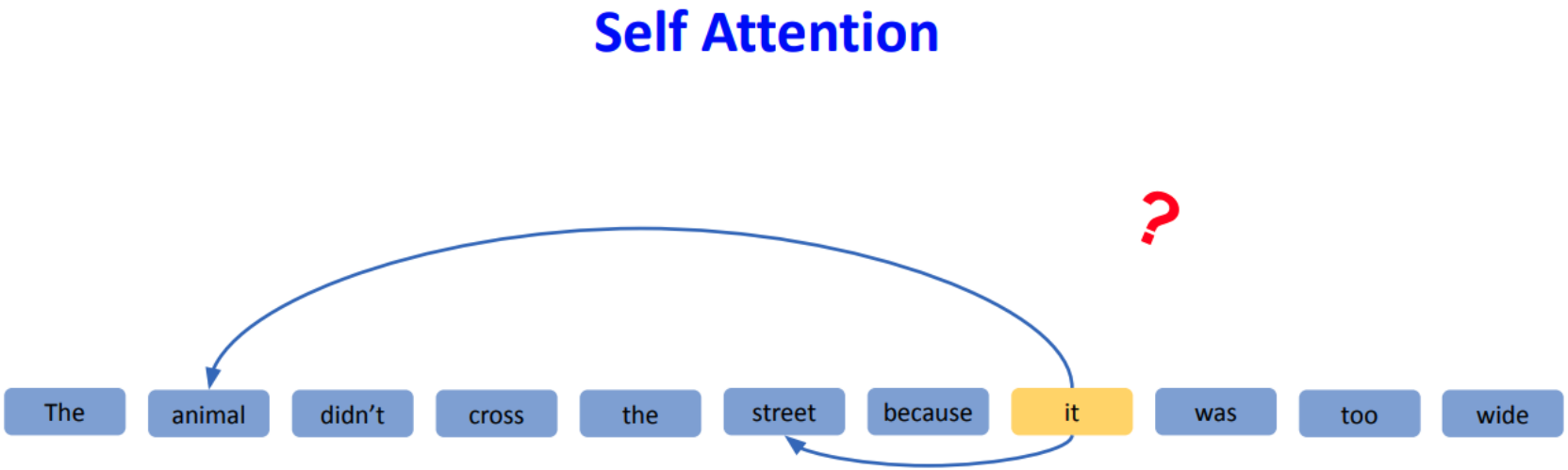
- Query
- Key
- Value



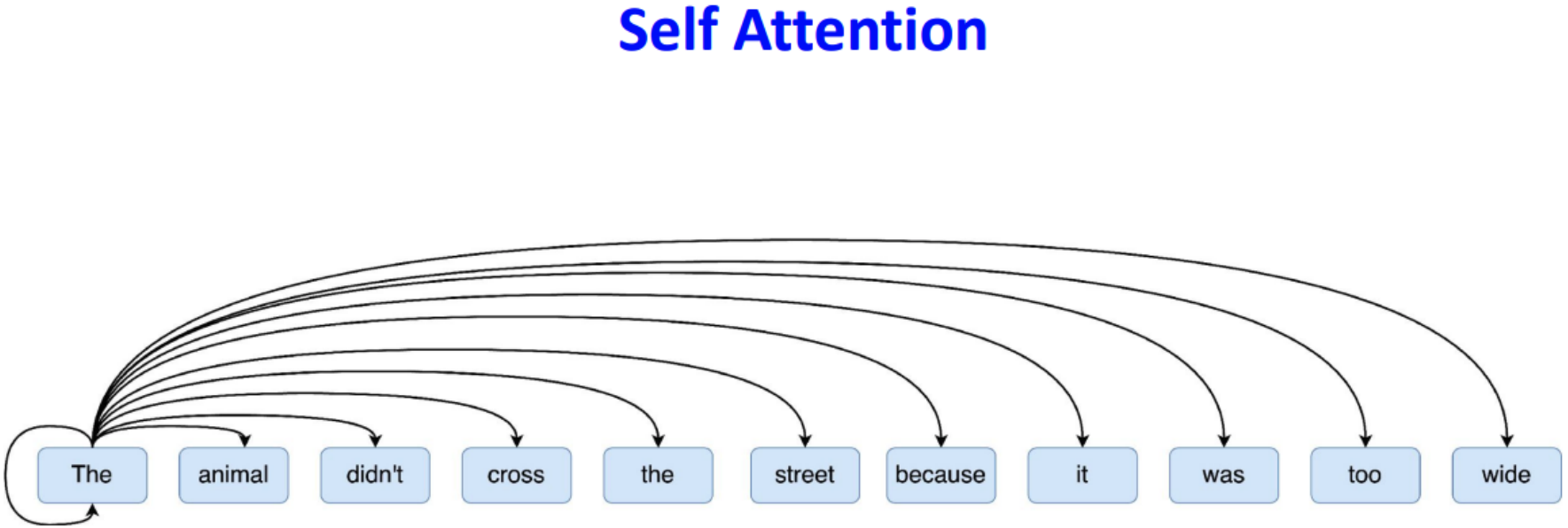
Self-Attention



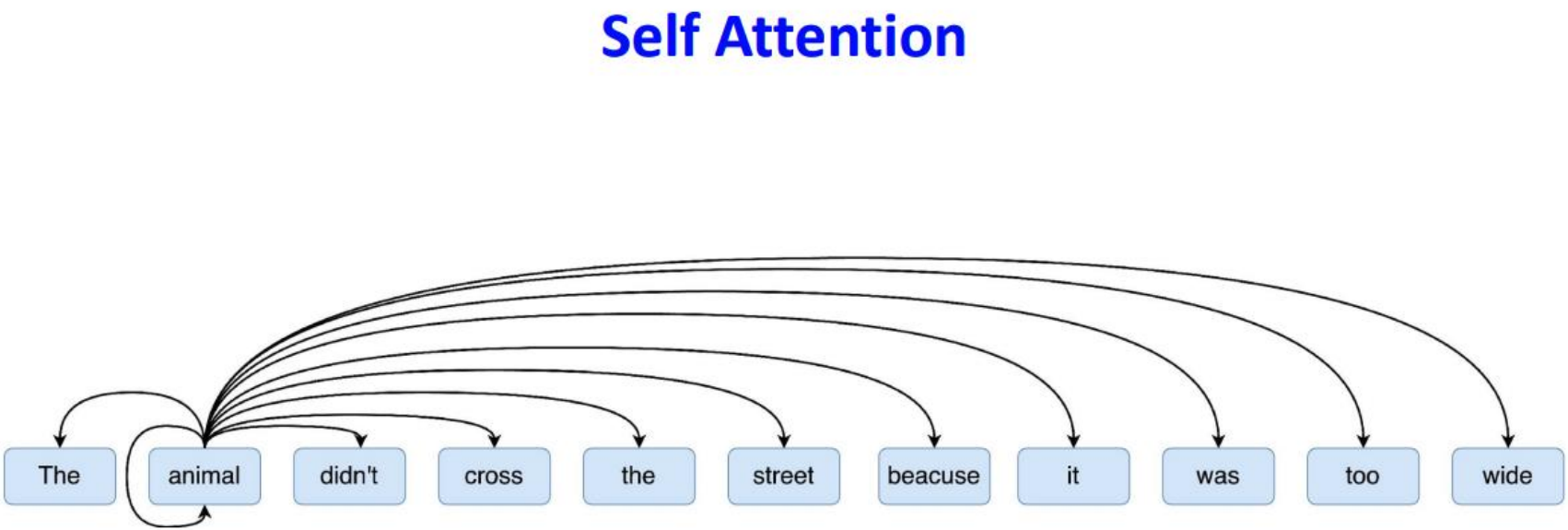
Self-Attention



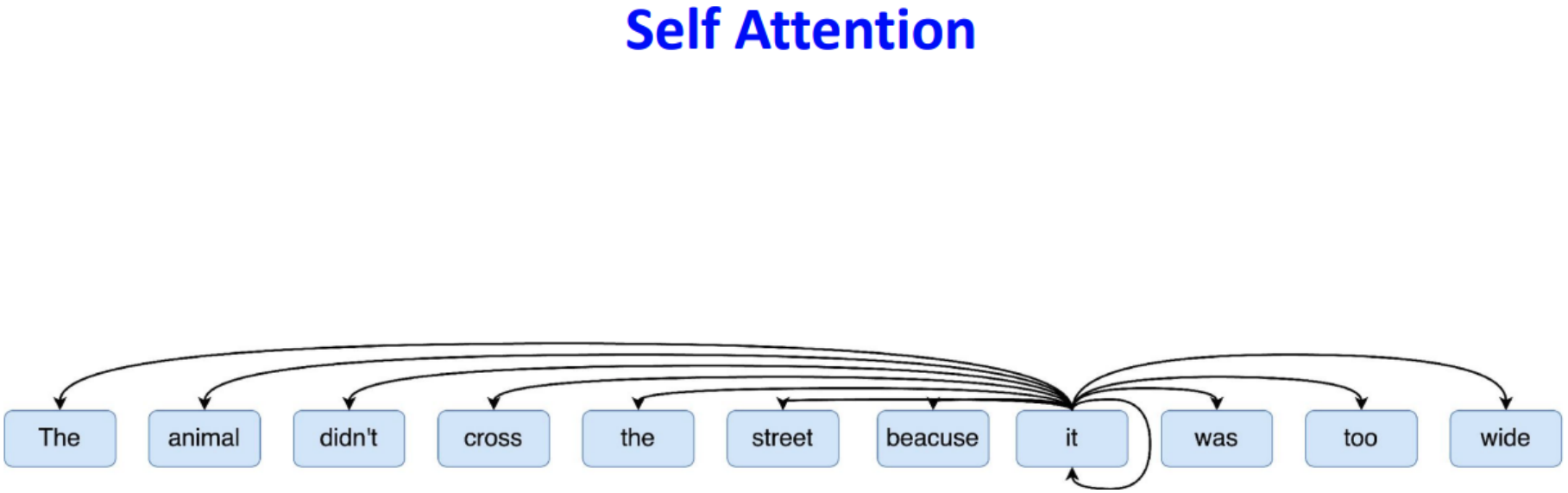
Self-Attention



Self-Attention

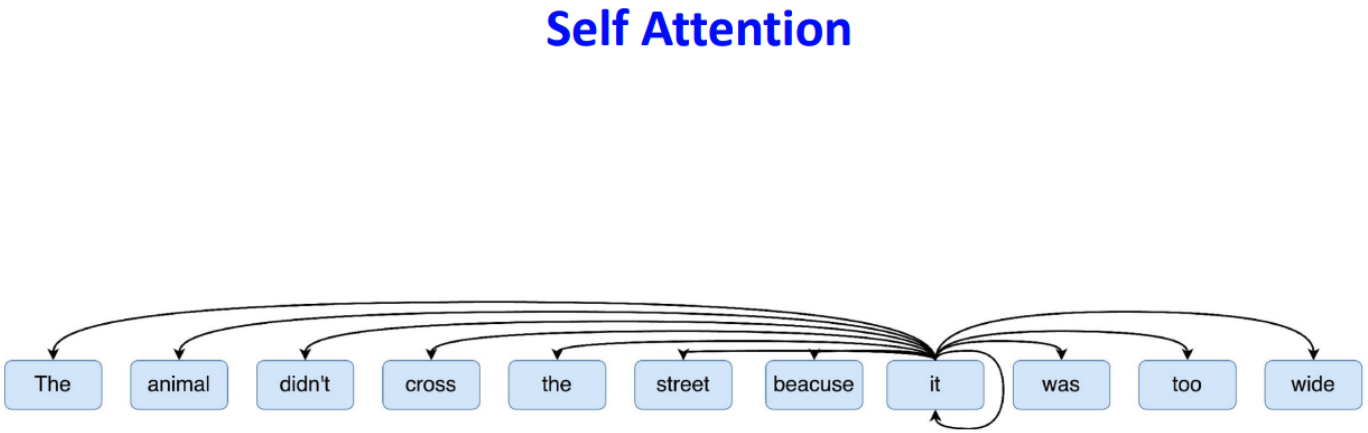


Self-Attention



The model tries to understand what **“it”** refers to. Using **self-attention**, the token **“it”** can attend to earlier tokens like *animal* or *street* to determine the correct meaning.

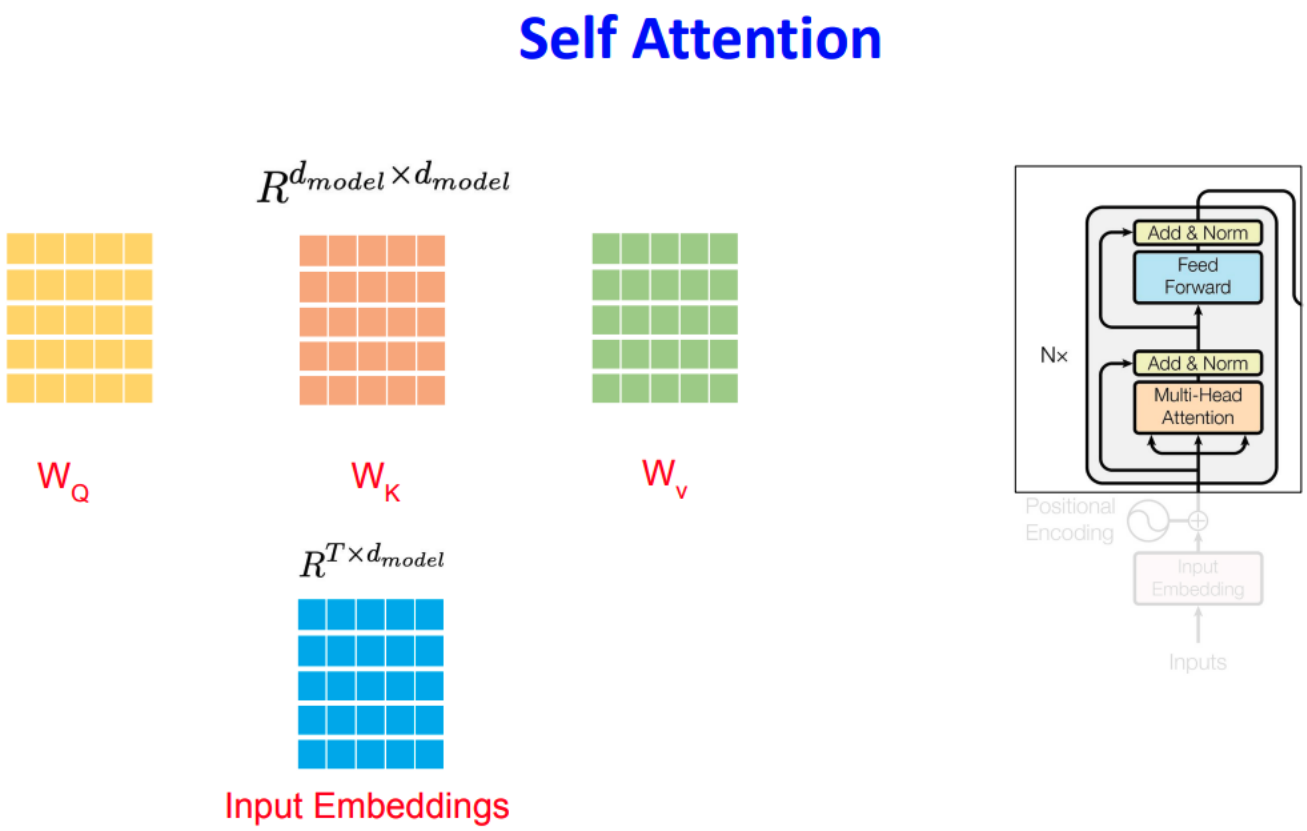
Self-Attention



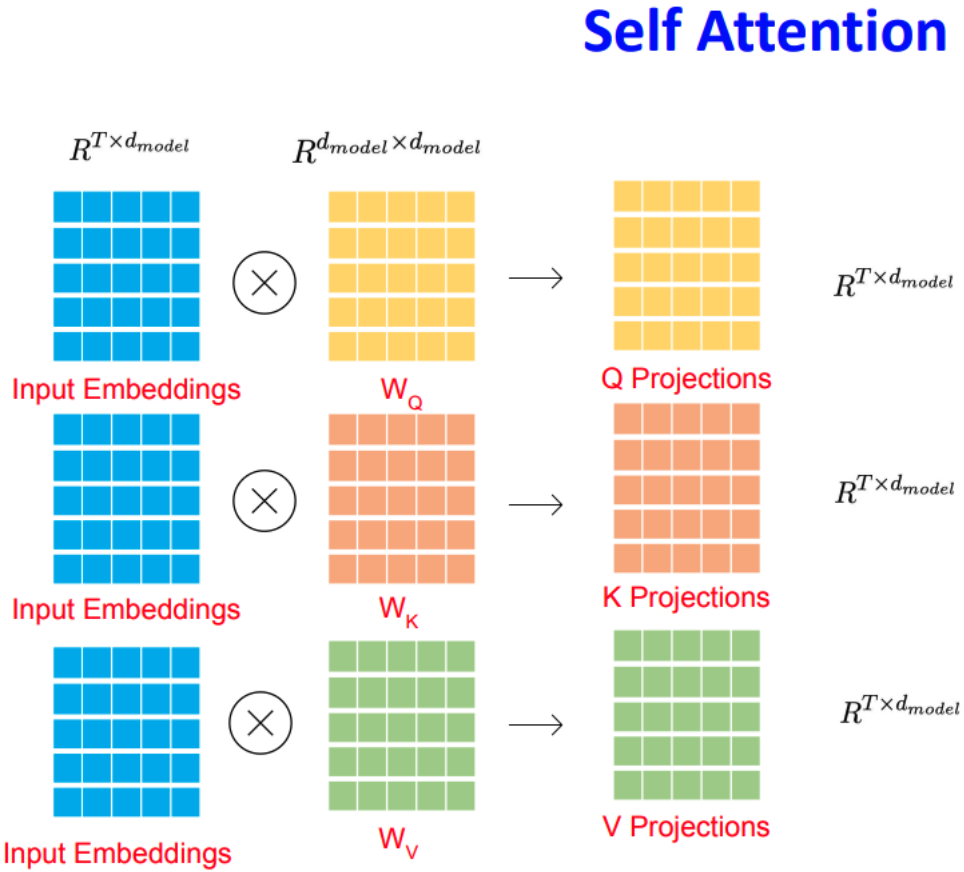
"Self" means:

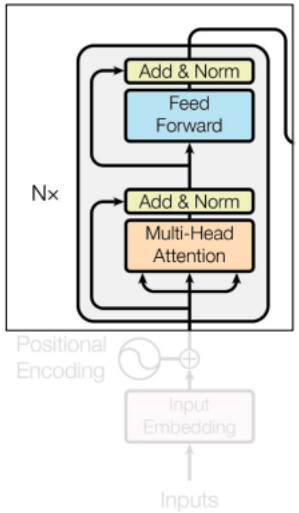
- The **queries, keys, and values all come from the same input sequence**.
- Every word can attend to **other words in the same sentence (including itself)**.

Self-Attention



Self-Attention

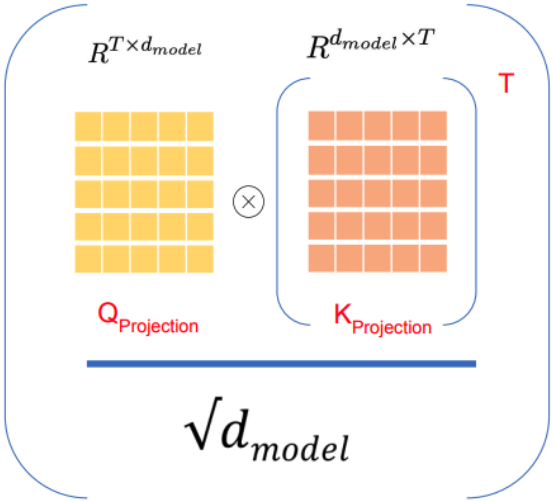




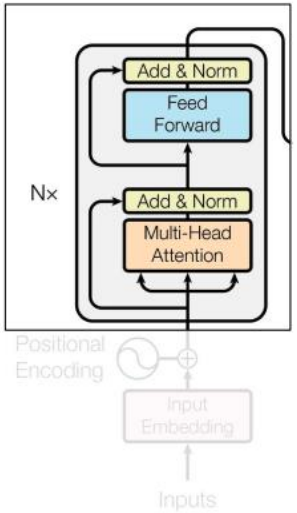
Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

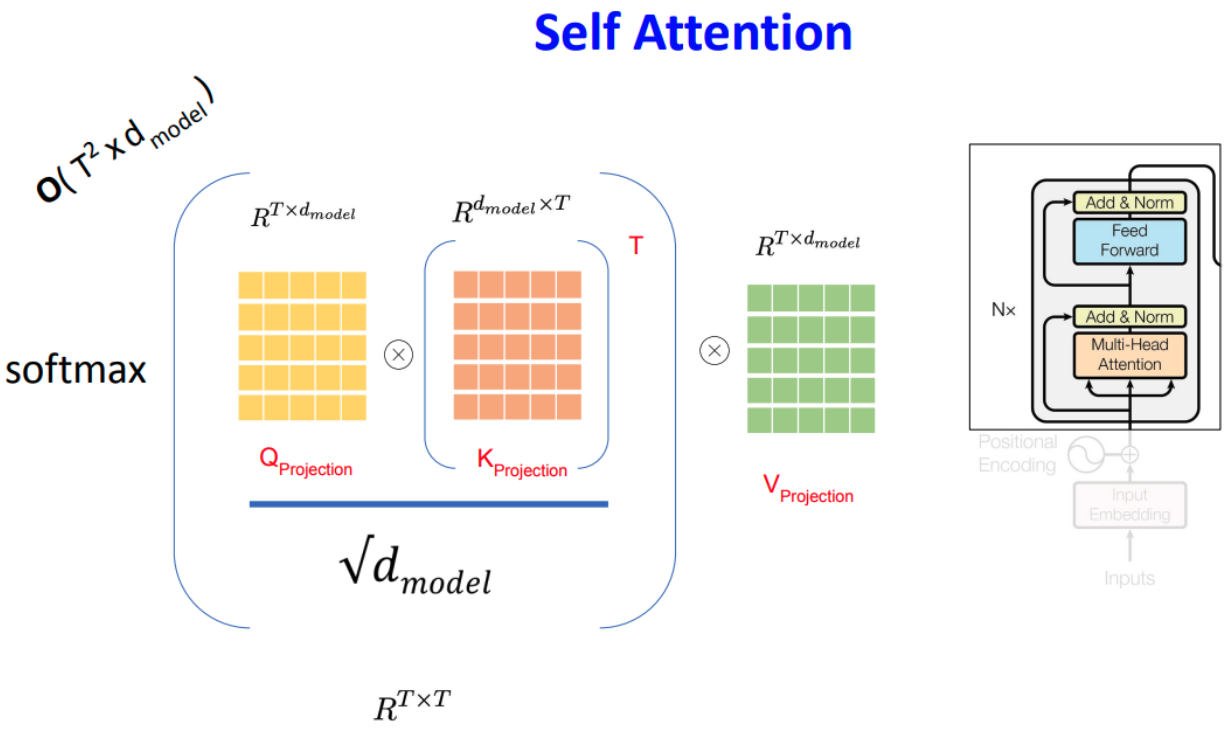
softmax



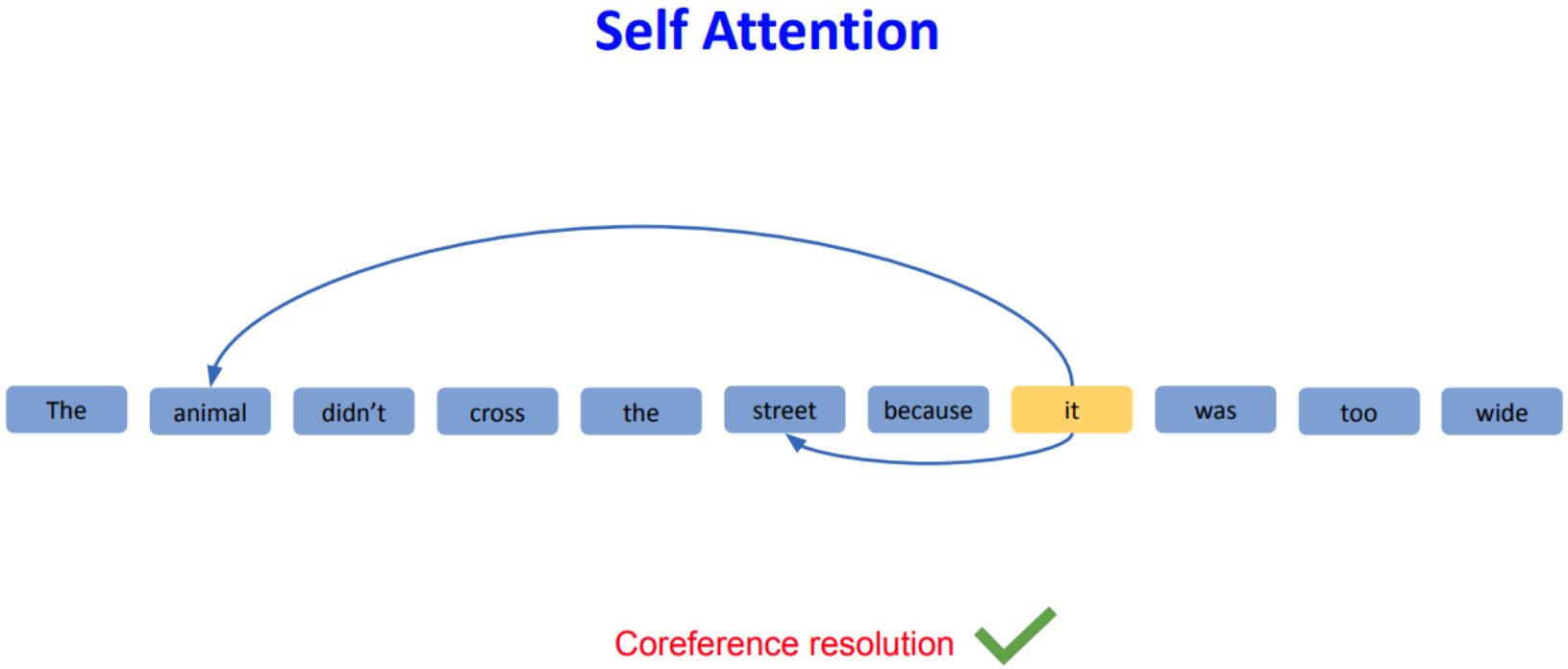
$R^{T \times T}$



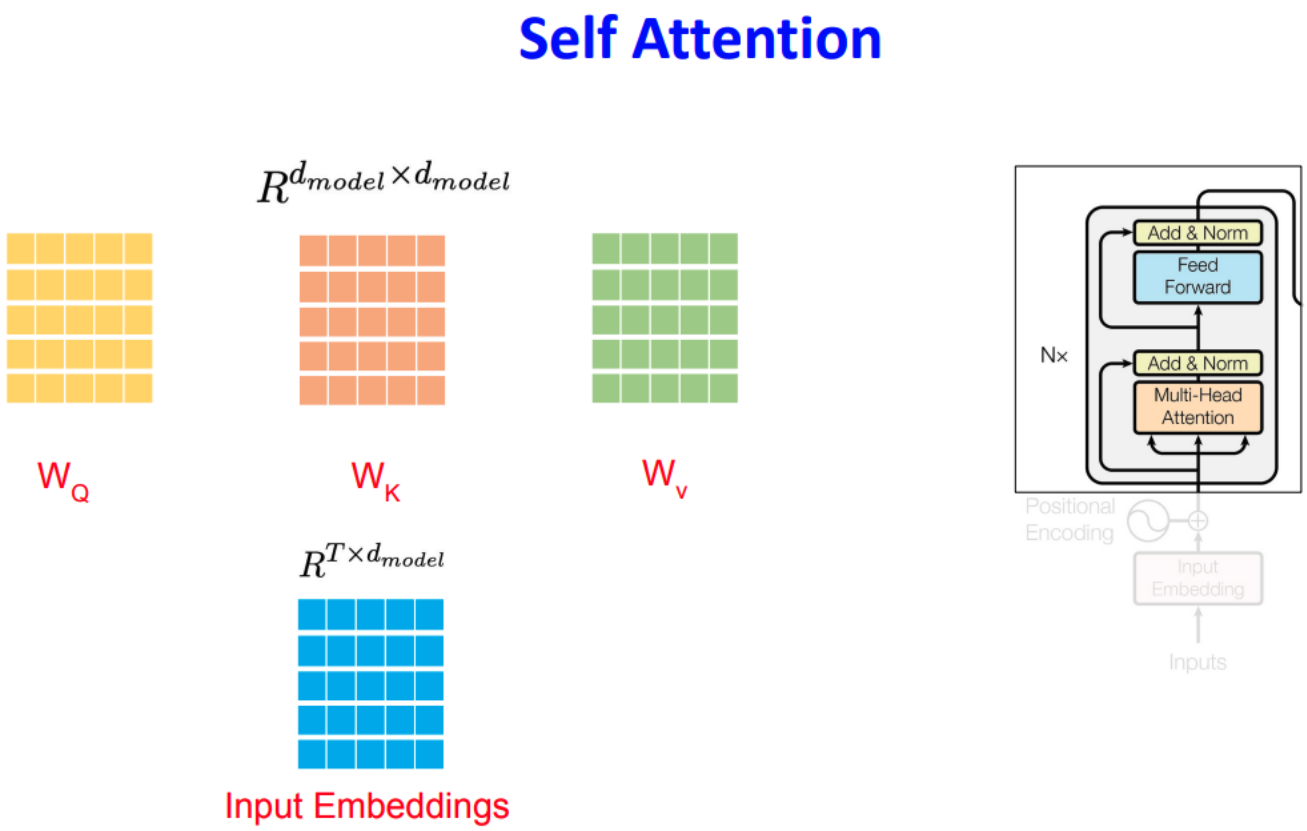
Self-Attention



Self-Attention

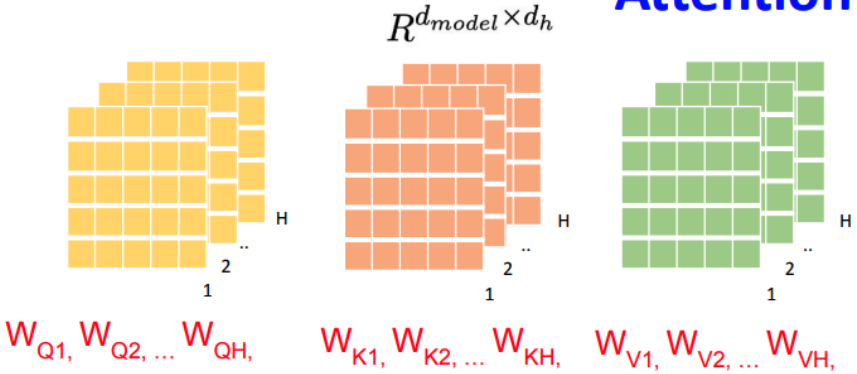


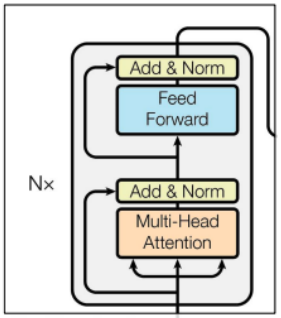
Self-Attention

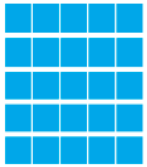


Self-Attention

Multi-Head Attention





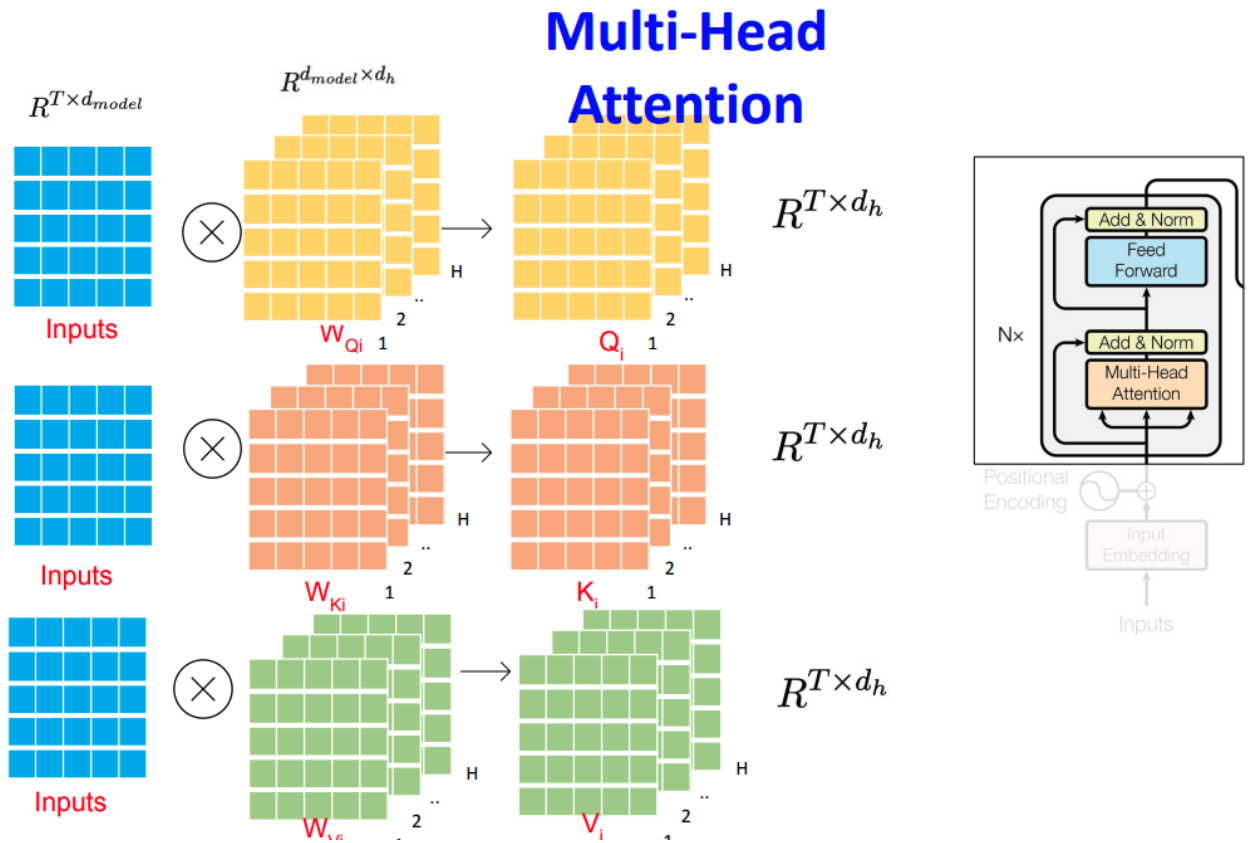


Input Embeddings

$R^{T \times d_{model}}$

$$d_h = \frac{d_{model}}{h}$$

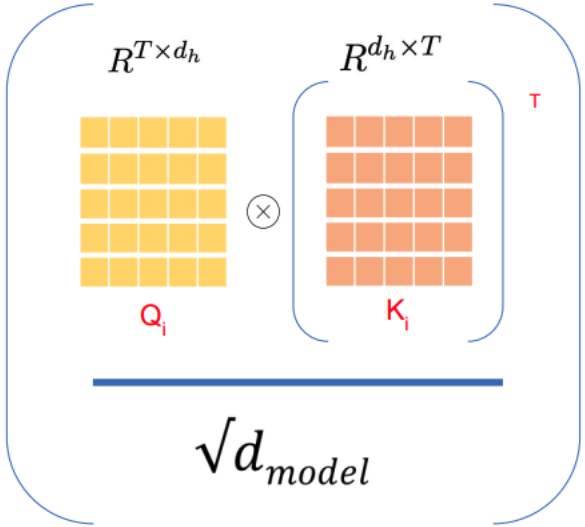
Self-Attention

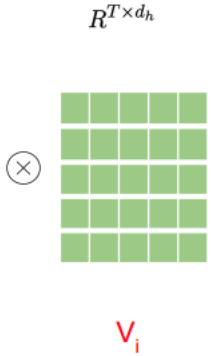


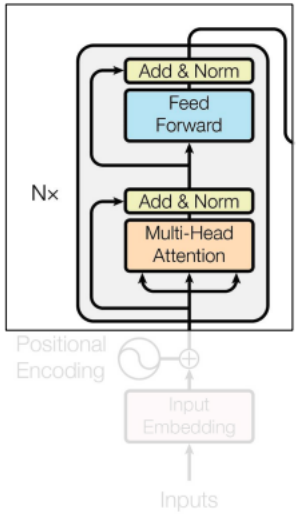
Self-Attention

Multi-Head Attention

softmax





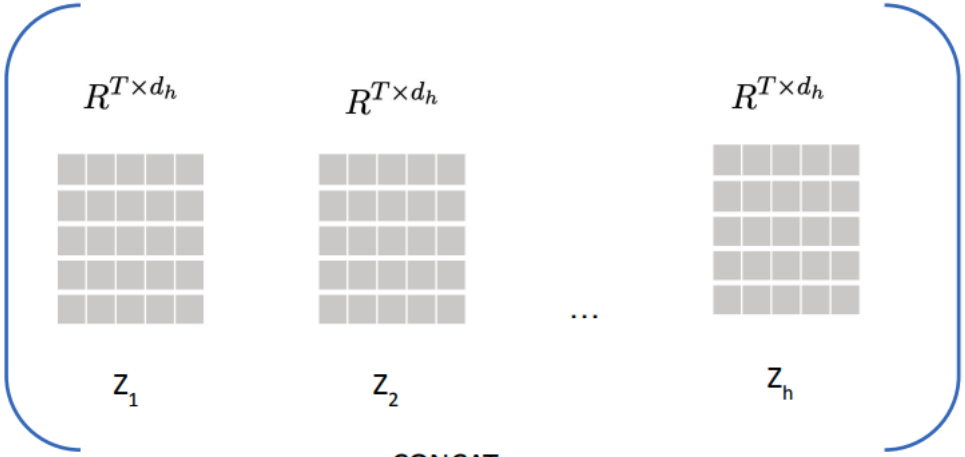


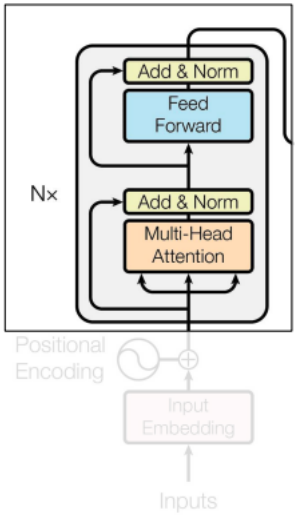
$R^{T \times T}$

for all i ∈ [1, h]

Self-Attention

Multi-Head Attention



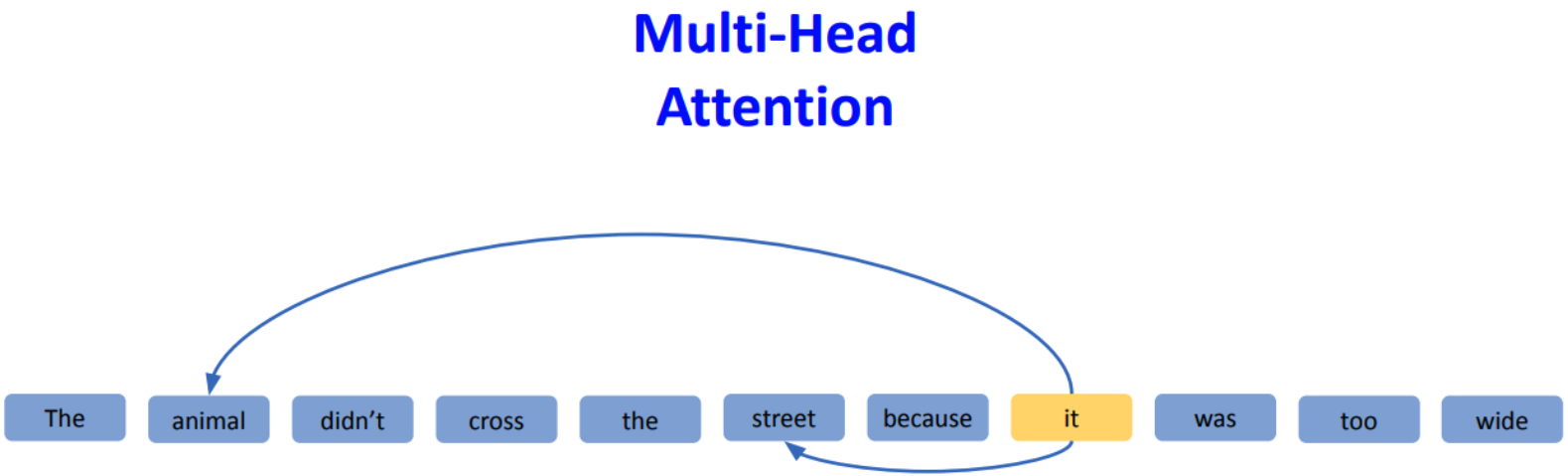


Multi Head Attention : Z

$$d_h = \frac{d_{model}}{h}$$

$$R^{T \times d_{model}}$$

Self-Attention

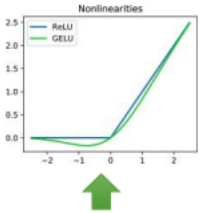


Sentence boundaries ✓ Coreference resolution ✓ Context ✓

Semantic relationships ✓ Part of speech ✓ Comparisons ✓

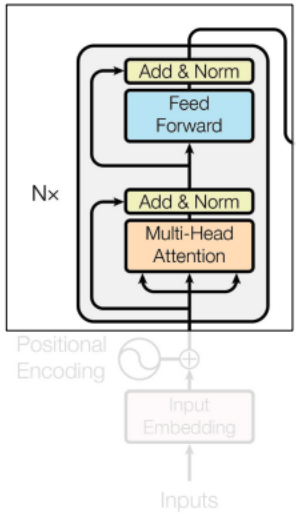
Feed Forward (MLP)

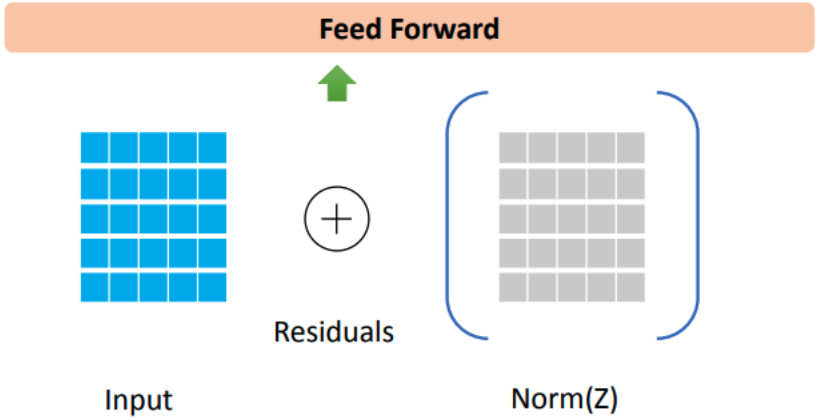
Feed Forward



Feed Forward

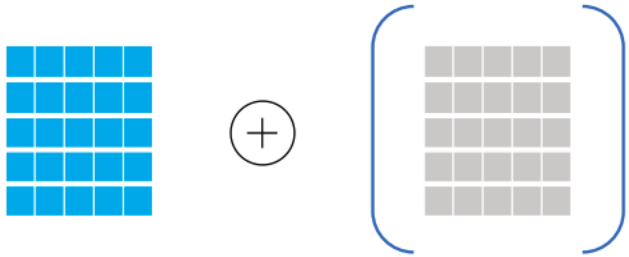
- Non Linearity
- Complex Relationships
- Learn from each other





Add & Norm

Add & Norm



Input

Norm(Z)

Normalization

Mean 0, Std dev 1

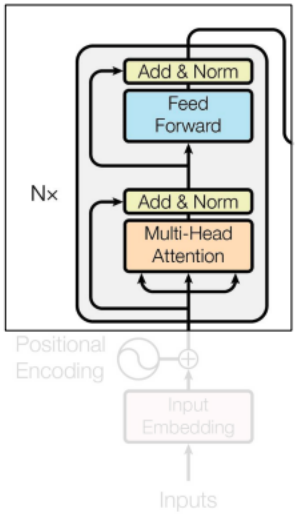
Stabilizes training

Regularization effect

Add Residuals

Avoid vanishing gradients

Train deeper networks



Transformer Encoders

Encoders

Encoder

ENCODER

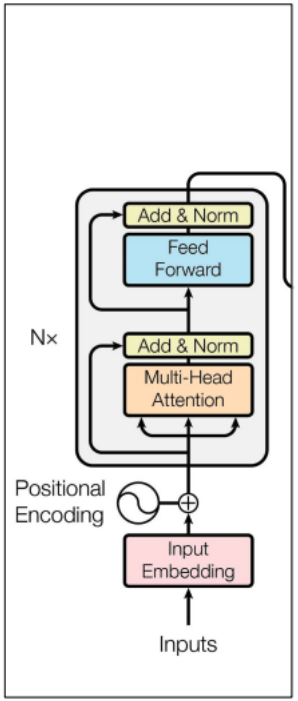
.
.
.

ENCODER

ENCODER

Input to Encoder$_{i+1}$

Output from Encoder$_{i}$

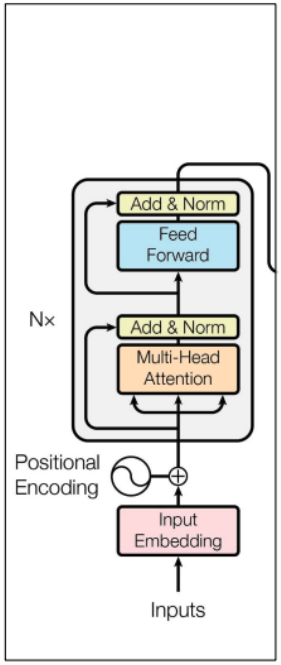


Transformers

Transformers

- ✔ Tokenization
- ✔ Input Embeddings
- ✔ Position Encodings
- ✔ Query, Key, & Value
- ✔ Attention
- ✔ Self Attention
- ✔ Multi-Head Attention
- ✔ Feed Forward
- ✔ Add & Norm
- ✔ Encoders

- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



Vision Transformers (ViT) use essentially the **Transformer encoder architecture**.

MLP Coding Practice

https://colab.research.google.com/drive/1wbOaBvac9zyt5Ho0wPUGcr94A1M1I4na?usp=sharing

